



UNIVERSIDADE FEDERAL DE MATO GROSSO
FACULDADE DE ARQUITETURA, ENGENHARIA E TECNOLOGIA
DEPARTAMENTO DE ENGENHARIA ELÉTRICA

ALEXANDRO CASTRO DA CONCEIÇÃO

**GERAÇÃO DE TENSÃO SENOIDAL COM SPWM E SVM PARA INVERSORES DE
FREQUÊNCIA TRIFÁSICOS COM ESP32 E MCPWM**

CUIABÁ – MT

2024

ALEXANDRO CASTRO DA CONCEIÇÃO

**GERAÇÃO DE TENSÃO SENOIDAL COM SPWM E SVM PARA INVERSORES DE
FREQUÊNCIA TRIFÁSICOS COM ESP32 E MCPWM**

Trabalho Final de Curso apresentado ao Departamento de Engenharia Elétrica da Universidade Federal de Mato Grosso, como requisito parcial para a obtenção do título de Bacharel em Engenharia Elétrica.

Orientador:
Prof. Dr. Jakson Paulo Bonaldo

CUIABÁ – MT

2024

Dados Internacionais de Catalogação na Fonte.

C744g Conceição, Alexandro Castro da.

Geração de tensão senoidal com SPWM e SVM para inversores de frequência trifásicos com ESP32 e MCPWM [recurso eletrônico] / Alexandro Castro da Conceição. -- Dados eletrônicos (1 arquivo : 90 f., il. color., pdf). -- 2024.

Orientador: Jakson Paulo Bonaldo.

TCC (graduação em Engenharia Elétrica) - Universidade Federal de Mato Grosso, Faculdade de Arquitetura, Engenharia e Tecnologia, Cuiabá, 2024.

Modo de acesso: World Wide Web: <https://bdm.ufmt.br>.

Inclui bibliografia.

1. MCPWM; Inversor de frequência; Microcontrolador ESP32; Modulação vetorial espacial; Modulação PWM senoidal.. I. Bonaldo, Jakson Paulo, *orientador*. II. Título.

Ficha catalográfica elaborada automaticamente de acordo com os dados fornecidos pelo(a) autor(a).

Permitida a reprodução parcial ou total, desde que citada a fonte.



UNIVERSIDADE FEDERAL DE MATO GROSSO
Av. Fernando Corrêa da Costa, n 2367 - Bairro Boa Esperança, Cuiabá/MT, CEP 78060-900
Telefone: (65)3615-8000 e Fax: @fax_unidade@ - <http://www.ufmt.br>

DECLARAÇÃO

Processo nº 23108.065126/2024-11

Interessado: Coordenação de Ensino de Graduação em Engenharia Elétrica

FOLHA DE APROVAÇÃO

TÍTULO DA MONOGRAFIA:

GERAÇÃO DE TENSÃO SENOIDAL COM SPWM E SVM PARA INVERSORES DE FREQUÊNCIA TRIFÁSICOS COM ESP32 E MCPWM.

Aluno: Alexandro Castro da Conceição

Trabalho de Conclusão de Curso apresentado à Faculdade de Arquitetura, Engenharia e Tecnologia da Universidade Federal de Mato Grosso, como requisito para a obtenção de grau de bacharel em Engenharia Elétrica.

Aprovada em 11 de setembro de 2024.

Nota: **9,43**

BANCA EXAMINADORA:

Prof. Nome-do-orientador - Prof. Dr. Jakson Paulo Bonaldo

Prof. Nome-do(a)-examindor(a)-1 - Prof. Dr. José Matheus Rondina

Prof. Nome-do(a)-examindor(a)-2 - Prof. Dr. Nicolás Eusebio Cortez Ledesma



Documento assinado eletronicamente por **JAKSON PAULO BONALDO**, Docente da Universidade Federal de Mato Grosso, em 11/09/2024, às 16:14, conforme horário oficial de Brasília, com fundamento no § 3º do art. 4º do [Decreto nº 10.543, de 13 de novembro de 2020](#).



Documento assinado eletronicamente por **NICOLAS EUSEBIO CORTEZ LEDESMA**, **Docente da Universidade Federal de Mato Grosso**, em 11/09/2024, às 17:10, conforme horário oficial de Brasília, com fundamento no § 3º do art. 4º do [Decreto nº 10.543, de 13 de novembro de 2020](#).



Documento assinado eletronicamente por **JOSE MATEUS RONDINA**, **Docente da Universidade Federal de Mato Grosso**, em 11/09/2024, às 19:20, conforme horário oficial de Brasília, com fundamento no § 3º do art. 4º do [Decreto nº 10.543, de 13 de novembro de 2020](#).



A autenticidade deste documento pode ser conferida no site http://sei.ufmt.br/sei/controlador_externo.php?acao=documento_conferir&id_orgao_acesso_externo=0, informando o código verificador **7156128** e o código CRC **A768396B**.

AGRADECIMENTOS

Agradeço,

Ao meu orientador Prof. Dr. Jakson Paulo Bonaldo pela valorosa orientação e ensino para que esse trabalho fosse possível.

Aos meus colegas de curso pelos anos juntos.

Aos amigos que fiz no curso de engenharia elétrica: Amanda, Ana Vitória, Raul, Samila, Wallison que estiveram ao meu lado nessa jornada, com muitas risadas e dedicação nos estudos.

A minha família, pelo apoio e incentivo que sempre me deram para que não desistisse dos meus sonhos.

A Vivianny pela sua valiosa amizade.

A minha amada mãe, Maria Aparecida Santos de Oliveira, sem a qual eu jamais teria chegado tão longe, grato sou pela sua vida e pelo tanto que ajudou me amando, aconselhando e dando subsídio para que isso fosse possível.

Principalmente, agradeço a Deus por ter me guiado até aqui.

RESUMO

Este trabalho aborda o uso do microcontrolador ESP32 para aplicar estratégias de geração de PWM para inversores de frequência no acionamento de máquinas de indução trifásicas. São apresentadas as estratégias de modulação utilizadas: SPWM e o SVM. O controle é efetivado por meio do ESP32 a partir de seu periférico MCPWM dedicado a operar chaveamento de transistores com foco em eletrônica de potência, como pontes H presente em inversores. A aplicação microcontrolada apresenta um menu com display LCD, botões para o usuário configurar o tipo de modulação manualmente e potenciômetro para modificar a frequência de rotação do motor em malha aberta, também conhecida como controle V/F. O ESP32 foi configurado com frequência de chaveamento de 20 kHz para gerar sinais senoidais de frequências entre 6 Hz e 120 Hz, sendo ajustável em tempo de execução, isto é, enquanto o inversor está operando. Toda a programação do microcontrolador foi realizada a partir do framework ESP-IDF em linguagem C. Neste trabalho foi possível acionar um motor a partir de um inversor e alterar sua velocidade de rotação do eixo usando as estratégias de modulação implementadas no microcontrolador.

Palavras-chave: MCPWM, Inversor de frequência, Microcontrolador ESP32, Modulação vetorial espacial, Modulação PWM senoidal.

ABSTRACT

This work addresses the use of the ESP32 microcontroller to apply PWM generation strategies for frequency inverters in the drive of three-phase induction machines. The modulation strategies used are presented: SPWM and SVM. The control is effective through the ESP32 from its MCPWM peripheral dedicated to operating transistor switching with a focus on power electronics, such as H-bridges present in inverters. The microcontroller application presents a menu with LCD display, buttons for the user to configure the modulation type manually and a potentiometer to modify the motor rotation frequency in open loop, also known as V/F control. The ESP32 was configured with a switching frequency of 20 kHz to generate sinusoidal signals of frequencies between 6 Hz and 120 Hz, being adjustable at runtime, that is, while the inverter is operating. All microcontroller programming was performed using the ESP-IDF framework in C language. In this work, it was possible to drive a motor from an inverter and change its rotation frequency during execution with the modulation generated by the microcontroller.

Keywords: MCPWM. Frequency inverter. ESP32. Space vector modulation. Sinusoidal PWM.

LISTA DE ILUSTRAÇÕES

Figura 1 - Representação gráfica da transformação de coordenadas abc para $\alpha\beta$	14
Figura 2 - Sistema trifásico equilibrado apresentado no referencial: a) abc, b) $\alpha\beta$ invariante em amplitude	17
Figura 3 - Implementação analógica de um PWM: a) Modulação PWM, b) Sinal modulado, c) Comparador analógico.....	17
Figura 4 - Circuito de um inversor monofásico ponte completa	18
Figura 5 - Padrão de onda de um inversor.....	19
Figura 6 – Forma de onda: a) modulação do SPWM, b) sinal modulado do SPWM	20
Figura 7 - SPWM com baixa frequência de chaveamento	21
Figura 8 - SPWM com alta frequência de chaveamento	21
Figura 9 - Circuito de um inversor trifásico	22
Figura 10 - Vetores de tensão de saída do inversor trifásico.....	23
Figura 11 - Geração do vetor de referência de tensão por superposição de vetores de saída do inversor	24
Figura 12 - Padrão de chaveamento do SVM.....	25
Figura 13 - Referenciais dos vetores espaciais.....	26
Figura 14 - Fluxograma do algoritmo de identificação do setor	28
Figura 15 - Forma de onda do sinal modulante gerada pelo SVM.....	29
Figura 16 - Máxima amplitude de tensão gerada pelo SVM.....	29
Figura 17 - Modelo equivalente simplificado do MIT em regime permanente.....	30
Figura 18 - Perfil de tensão no estator versus frequência em modo de controle escalar.....	31
Figura 19 - Diagrama de blocos do controle V/F em malha aberta.....	32
Figura 20 - Visão geral do MCPWM	33
Figura 21 - Diagrama de blocos de uma unidade MCPWM	35
Figura 22 - PWM digital: a) organização, b) exemplo de um PWM digital	36
Figura 23 - PWM digital uniformemente amostrado: a) diagrama de blocos geral, b) modulação com portadora triangular	37
Figura 24 - PWM digital do módulo MCPWM do ESP32.....	38
Figura 25 - Chaveamento de um transistor: a) para condução, b) para corte	39
Figura 26 - Opções para configurar o submódulo do gerador de tempo morto do ESP32.....	40
Figura 27 - Tempo morto ativo alto complementar.....	41
Figura 28 - Fluxograma do processamento do algoritmo.....	44
Figura 29 - Configuração do MCPWM timer e MCPWM operator	45

Figura 30 - Configuração do MCPWM comparator e MCPWM generator	45
Figura 31 - Configuração das ações do MCPWM generator e do dead time	46
Figura 32 - Configuração da função de interrupção	47
Figura 33 - Função de interrupção para atualização do sinal de modulação	48
Figura 34 - Circuito de acionamento do inversor e configuração do tipo de modulação	49
Figura 35 - Ajuste da frequência	50
Figura 36 - Circuito de controle e de potência	50
Figura 37 - Esquemático do inversor: a) sistema de potência, b) sistema microcontrolado	51
Figura 38 – Forma de onda da saída dos PWM's de um braço: a) Tensão entre PWM superior e inferior de um braço, b) tempo morto	52
Figura 39 - Forma de onda característica do SVM.....	53
Figura 40 - Sinal PWM: a) tempo morto ativo complementar baixo gerado pelo ESP32, b) tempo morto ativo complementar alto da placa inversora.....	54
Figura 41 – Forma de onda do PWM trifásico simétrico gerado pelo SVM.....	55
Figura 42 – Forma de onda da tensão V_{ab} a 4 ms/div com SVM.....	55
Figura 43 – Forma de onda da corrente elétrica no motor com o SPWM.....	56
Figura 44 – Forma de onda da corrente elétrica no motor com SVM.....	57
Figura 45 - Corrente elétrica no motor com SVM a 250 VDC	57

LISTA DE TABELAS

Tabela 1 - Período de modulação das chaves para cada setor dos vetores espaciais	28
Tabela 2 - Dados do motor utilizado	42

LISTA DE ABREVIATURAS E SIGLAS

<i>MCPWM</i>	<i>Motor Control Pulse Width Modulation</i> - Controle de motor por modulação de largura de pulso
<i>DHT</i>	Distorção Harmônica Total.
<i>SVM</i>	<i>Space Vector Modulation</i> - Modulação Vetorial Espacial
<i>FACTS</i>	<i>Flexible Alternating Current Transmission System</i> - Sistemas de Transmissão Flexíveis em Corrente Alternada
<i>SPWM</i>	<i>Sinusoidal Pulse Width Modulation</i> - Modulação por Largura de Pulso Senoidal
<i>MIT</i>	Motor de Indução Trifásico
<i>CA</i>	Corrente Alternada
<i>CC</i>	Corrente Contínua
<i>DC</i>	<i>Direct Current</i> – Corrente Contínua
<i>LCD</i>	<i>Liquid Crystal Display</i> - Display de cristal líquido
<i>DSP</i>	<i>Digital Signal Processor</i> – processador digital de sinal

1. INTRODUÇÃO.....	12
1.1. Problemática	12
1.2. Justificativa.....	13
1.3. Objetivos.....	13
1.3.1. Objetivo Geral	13
1.3.2. Objetivos Específicos	13
2. REFERENCIAL TEÓRICO.....	14
2.1. Transformada de Clarke ($abc - \alpha\beta$)	14
2.1.1. Dedução matemática.....	15
2.2. Modulação por largura de pulso (PWM).....	17
2.3. Inversor monofásico	18
2.4. Inversor trifásico.....	22
2.5. Modulação vetorial espacial	22
2.5.1. Algoritmo de implementação do SVM.....	25
2.6. Controle escalar V/F do motor de indução trifásico.....	30
2.7. Microcontrolador ESP32	33
2.7.1. Módulo MCPWM do ESP32.....	33
2.7.2. PWM digital com MCPWM do ESP32.....	35
2.8. Tempo morto	39
2.8.1. Tempo morto com MCPWM do ESP32.....	40
2.9. IDE e framework de programação.....	41
3. MATERIAIS E MÉTODOS.....	42
3.1. Materiais	42
3.2. Método.....	42
3.2.1. Fluxograma do algoritmo	43
3.2.2. Inicialização do MCPWM	44
4. RESULTADOS	49

4.1.	Implementação do sistema digital de acionamento do inversor trifásico	49
4.1.1.	Protótipo do circuito de acionamento	49
4.2.	Resultados experimentais	51
5.	CONCLUSÕES	58
5.1.	Sugestões para trabalhos futuros	58
	REFERÊNCIAS	59
	ANEXOS	61
	ANEXO A – Código principal utilizado para programação da modulação do inversor	61
	ANEXO B – Parte 1 da biblioteca criada para o código do algoritmo do SVM e SPWM ..	84
	ANEXO C - Parte 2 da biblioteca criada para o código do algoritmo do SVM e SPWM ...	86

1. INTRODUÇÃO

A eletrônica de potência é o campo da ciência que estuda e desenvolve dispositivos eletrônicos capazes de processar e transformar energia elétrica com o máximo de qualidade e eficiência possível. Seu principal objetivo é controlar o fluxo de energia elétrica, com a utilização de dispositivos semicondutores (diodos, tiristores e transistores) operando como chaves (ligar/desligar), realizando a conversão de formas de onda de tensão e corrente para fontes e cargas com o mínimo de perdas.

A eletrônica de potência já se encontra presente em diversas áreas do setor elétrico, como na transmissão de energia com os sistemas flexíveis de transmissão em corrente alternada, do inglês *flexible alternating current transmission system* - FACTS como por exemplo compensadores estáticos; nas residências e comércios com eletrodomésticos e computadores que possuem fontes chaveadas como conversores de nível de tensão do tipo corrente contínua para corrente contínua (CC-CC); na geração distribuída com placas fotovoltaicas e inversores conectados à rede elétrica e na indústria com retificadores, soft-starters e inversores de frequência para acionamento e controle de motores elétricos.

No quadro nacional, a indústria é o setor que mais consome energia elétrica e a maior parte dessa energia é utilizada em motores elétricos que são destinados a sistemas de bombeamento de água, ventilação, compressão, içamento e transporte de cargas por exemplo. Portanto, desenvolver sistemas simples e baratos com a eletrônica de potência que possam controlar, processar e transformar a energia para minimizar o consumo de energia pelo motor e maximizar sua operação são alvos de pesquisa acadêmica e do setor industrial.

1.1. Problemática

No contexto da indústria, onde o motor de indução trifásico é amplamente utilizado nos mais diversos tipos de cargas, há casos em que é necessário manter a velocidade do eixo do motor e o torque constantes, ou seja, controlar essas variáveis enquanto o motor está em funcionamento. Uma maneira de realizar tal tarefa é utilizando inversor de frequência para poder alterar de maneira dinâmica a tensão e a frequência na qual o motor está trabalhando e, conseqüentemente, controlar a velocidade e o torque.

O inversor de frequência é um dispositivo baseado em eletrônica de potência que converte corrente contínua em alternada (conversor CC-CA), sendo composto por componentes semicondutores de potência, tais como transistores, diodos e um sistema microcontrolado para acionar o inversor e controlar o motor.

1.2. Justificativa

Este trabalho busca preencher uma lacuna encontrada na literatura, que consiste em utilizar o microcontrolador ESP32 e seu periférico de controle de motor por largura de pulso, do inglês *Motor Control Pulse Width Modulation* – MCPWM ainda pouco explorado em textos acadêmicos para o acionamento e controle de motores com inversor de frequência. O MCPWM ainda é um periférico pouco explorado desse microcontrolador, diferente de outros como o Arduino Mega, carecendo então de projetos acadêmicos a respeito do ESP32 para a eletrônica de potência.

1.3. Objetivos

1.3.1. Objetivo Geral

Este trabalho tem como objetivo a análise e implementação das técnicas de geração de tensão alternada através do chaveamento de inversores utilizando o microcontrolador ESP32 para controle dos braços do inversor trifásico, acionando um motor de indução trifásico e ajustando a velocidade do eixo do motor. Além disso, será realizado um sistema para ajustar a tensão e frequência do motor.

1.3.2. Objetivos Específicos

- Estudar os princípios da conversão CC/CA com o inversor;
- Estudar o funcionamento do SPWM (*sinusoidal pulse width modulation*) e o SVM (*space vector modulation*);
- Estudar os periféricos do microcontrolador ESP32 que são essenciais para a implementação das técnicas de acionamento do inversor;
- Realizar um estudo do funcionamento e modelagem das máquinas de indução trifásicas para entender seus parâmetros e realizar seu controle;
- Implementar na prática um inversor de frequência que possa ajustar a velocidade de um motor de indução trifásico;

2. REFERENCIAL TEÓRICO

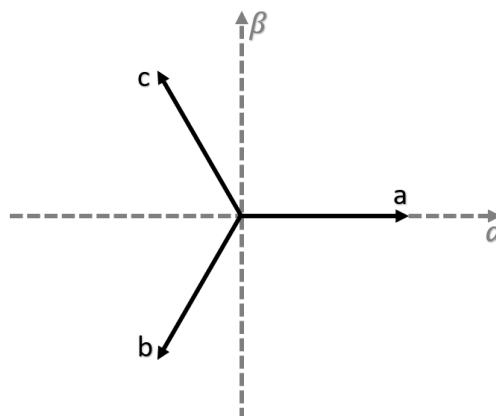
Nesta seção os assuntos tratados serão sobre a transformada de Clarka, fundamental para análise de sistemas trifásicos; modulação por largura de pulso, do inglês *Pulse Width Modulation* – PWM. Inversor monofásico e modulação por largura de pulso senoidal, do inglês *Sinusoidal Pulse Width Modulation* – SPWM. Inversor trifásico e modulação vetorial espacial, do inglês *Space Vector Modulation* – SVM. Periférico MCPWM do ESP32 e por fim, controle V/F em malha aberta para motores de indução.

2.1. Transformada de Clarke ($abc - \alpha\beta$)

A transformada de Clarke, também conhecida como transformada $\alpha\beta$ é utilizada para o estudo de sistemas elétricos trifásicos. “Em 1938, Edith Clarke apresentou uma modificação das componentes simétricas a fim de remover a parte imaginária e facilitar os cálculos” (GURGEL, 2021, p. 2), dando origem à transformada de Clarke.

Para explicar seu funcionamento podemos considerar um vetor tridimensional que pode representar qualquer triplete das variáveis elétricas do sistema (tensões ou correntes). Essas variáveis são representadas em referencial natural, que possui três eixos defasados de 120° , os eixos das fases a , b e c . Os vetores do referencial abc podem ser representados em dois eixos defasados de 90° decompondo seus vetores nesses novos eixos, α e β , conhecido como referencial de Clarke. Utilizando a notação de números complexos, o eixo α é escolhido como o eixo real (Re) e o eixo β como o imaginário (Im) (BRITO, 2014). A Figura 1 exhibe como está disposto o sistema de coordenadas (referencial abc e referencial $\alpha\beta$).

Figura 1 - Representação gráfica da transformação de coordenadas abc para $\alpha\beta$



Fonte: O autor

A mudança de coordenadas é feita realizando a projeção de cada fase do referencial abc nos eixos α e β . Considerando o caso de tensões elétricas trifásicas, a transformação é dada por (1):

$$\begin{bmatrix} V_\alpha \\ V_\beta \\ V_\gamma \end{bmatrix} = \frac{2}{3} \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \end{bmatrix} \begin{bmatrix} V_a \\ V_b \\ V_c \end{bmatrix} \quad (1)$$

Tal relação matricial é conhecida como amplitude invariante (AI). V_γ é considerada a componente homopolar, ou de sequência zero do sinal original. Caso se realize a transformação para o referencial $\alpha\beta$ ao invés do referencial $\alpha\beta\gamma$, perde-se a informação sobre a componente homopolar. Isso pode ser verificado pelo fato de que ao somar valores iguais nas três fases e realizar a mudança de coordenadas para $\alpha\beta$, obtém-se o mesmo resultado. Por isso, para a transformação ficar completa, necessita-se da componente homopolar, proporcional à soma das três componentes de fase (BRITO, 2014). Contudo, essa transformação possui uma propriedade interessante, que fica clara quando se leva em consideração que:

$$V_a + V_b + V_c = 0 \Rightarrow V_\gamma = 0 \quad (2)$$

O que significa que para circuitos elétricos equilibrados, a componente γ pode ser desconsiderada, mantendo apenas dois eixos no sistema de coordenadas.

A operação inversa também pode ser feita, para obter as variáveis no referencial natural (abc) a partir das variáveis no referencial $\alpha\beta$ conforme (3).

$$\begin{bmatrix} V_a \\ V_b \\ V_c \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ -\frac{1}{2} & \frac{\sqrt{3}}{2} & 1 \\ -\frac{1}{2} & -\frac{\sqrt{3}}{2} & 1 \end{bmatrix} \begin{bmatrix} V_\alpha \\ V_\beta \\ V_\gamma \end{bmatrix} \quad (3)$$

2.1.1. Dedução matemática

A Transformação de Clarke foi inicialmente desenvolvida por uma modificação da transformada de Fortescue. Para uma apresentação mais detalhada de Fortescue, recomenda-se

verificar (FORTESCUE, 1918). Sendo assim, para um sistema de tensões trifásico equilibrado, em componentes simétricas tem-se:

$$V_{012} = \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & \alpha & \alpha^2 \\ 1 & \alpha^2 & \alpha \end{bmatrix} \begin{bmatrix} V \cos(\omega t) \\ V \cos(\omega t - 2\pi/3) \\ V \cos(\omega t + 2\pi/3) \end{bmatrix} = \frac{V}{2} \begin{bmatrix} 0 \\ \cos(\omega t) + j \sin(\omega t) \\ \cos(\omega t) - j \sin(\omega t) \end{bmatrix} \quad (4)$$

Onde, na matriz de Fortescue $\alpha = e^{\frac{j2\pi}{3}} = \cos(2\pi/3) + j \sin(2\pi/3)$. A partir de (4), Edith Clarke propôs sua transformada para eliminar as partes imaginárias, sendo esse cálculo dado por (5):

$$V_{\gamma\alpha\beta} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & -j & j \end{bmatrix} \frac{V}{2} \begin{bmatrix} 0 \\ \cos(\omega t) + j \sin(\omega t) \\ \cos(\omega t) - j \sin(\omega t) \end{bmatrix} = V \begin{bmatrix} 0 \\ \cos(\omega t) \\ \sin(\omega t) \end{bmatrix} \quad (5)$$

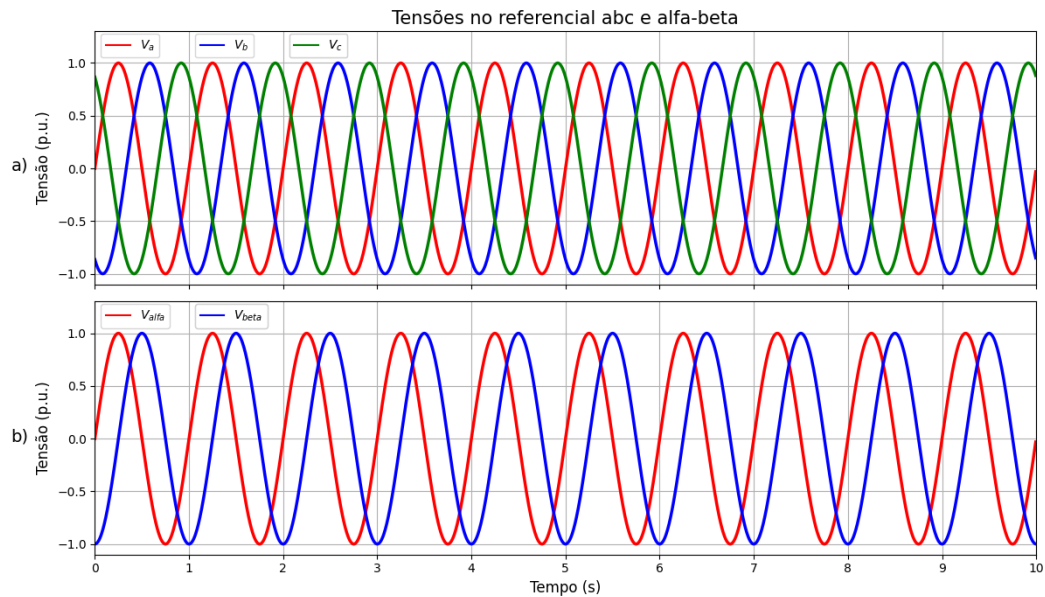
Relacionando diretamente o referencial abc com $\alpha\beta$, tem-se (6):

$$\begin{aligned} T_{CK} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & -j & j \end{bmatrix} \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & \alpha & \alpha^2 \\ 1 & \alpha^2 & \alpha \end{bmatrix} = \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 2 & \alpha + \alpha^2 & \alpha^2 + \alpha \\ 0 & -j(\alpha - \alpha^2) & j(\alpha^2 - \alpha) \end{bmatrix} \\ &= \frac{2}{3} \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \end{aligned} \quad (6)$$

As potências ativas e reativas computadas no domínio de Clarke com a transformação mostrada acima não são as mesmas computadas no referencial padrão. Isso acontece porque T_{CK} não é unitário. Para preservar a informação da potência, deve-se considerar um fator de multiplicação de $\sqrt{2/3}$ ao invés de $2/3$ e a matriz fica conhecida como transformação invariante em potência.

Um caso interessante para se ilustrar é o de um sistema trifásico equilibrado. Usando (1) para realizar a transformação de coordenadas, a Figura 2 ilustra as variáveis de interesse em cada sistema de coordenadas.

Figura 2 - Sistema trifásico equilibrado apresentado no referencial: a) abc, b) $\alpha\beta$ invariante em amplitude

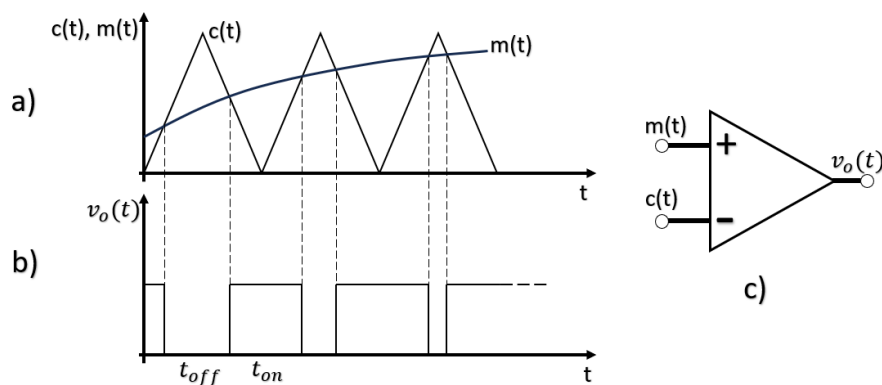


Fonte: O autor

2.2. Modulação por largura de pulso (PWM)

PWM é uma técnica de modulação na qual a duração ou largura dos pulsos modulados varia em proporção com a amplitude de um sinal de modulação (sinal modulante) em comparação com um sinal portador (KART, 2001), onde normalmente o sinal portador é uma onda do tipo dente de serra ou triangular. A modulação PWM é comumente utilizada em eletrônica de potência para acionamento de fontes chaveadas e em inversores. A Figura 3 mostra como funciona esse tipo de modulação.

Figura 3 - Implementação analógica de um PWM: a) Modulação PWM, b) Sinal modulado, c) Comparador analógico



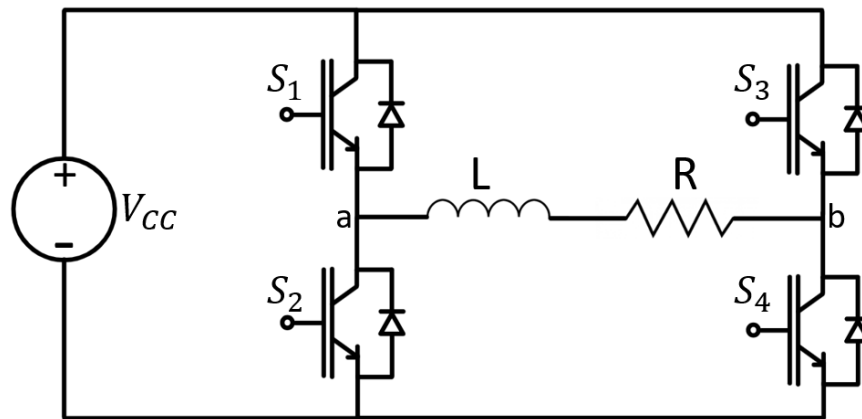
Fonte: O autor

A comparação entre o sinal modulante e o sinal portador na Figura 3(a) determina o estado da tensão de saída na Figura 3(b) comparando o sinal portador $c(t)$ e o sinal modulante $m(t)$, sendo realizada pelo modulador PWM mostrado na Figura 3(c).

2.3. Inversor monofásico

O inversor monofásico do tipo ponte completa é exibido na Figura 4, possuindo quatro chaves eletrônicas, dois pares complementares em cada braço, ou seja, as chaves S_1 e S_2 devem estar em estados opostos, bem como as chaves S_3 e S_4 .

Figura 4 - Circuito de um inversor monofásico ponte completa

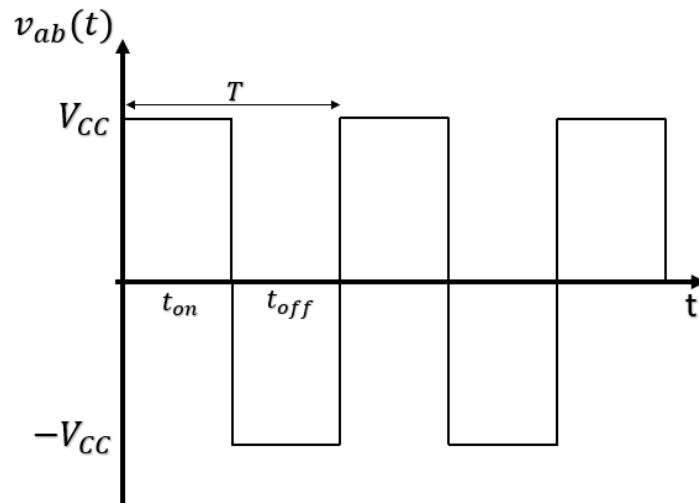


Fonte: O autor

Quando as chaves S_1 e S_4 estão ligadas e S_2 e S_3 desligados, a tensão v_{ab} aplicada nos terminais da carga será de V_{CC} . Na situação contrária, com S_2 e S_3 ligados, a tensão v_{ab} na carga será $-V_{CC}$.

No final de um período de chaveamento T , pode-se obter qualquer valor médio entre V_{CC} e $-V_{CC}$ para v_{ab} naquele período. A forma de onda típica desse inversor é descrita na Figura 5.

Figura 5 - Padrão de onda de um inversor

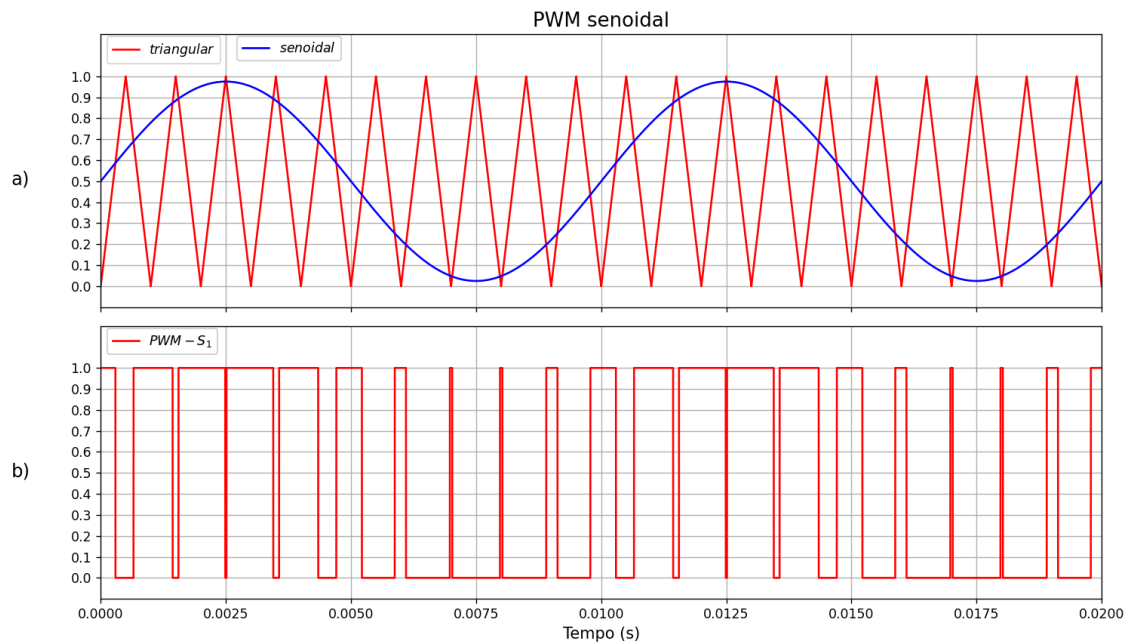


Fonte: O autor

A saída dessa onda é quadrada, ou seja, com conteúdo harmônico próximo da componente fundamental (SAENZ, 2021), o que não é desejável em aplicações de potência, onde deseja-se uma forma de onda o mais próximo possível da senoidal com baixa distorção harmônica.

Para obter uma onda o mais próximo possível de uma senoide, é realizada a modulação por largura de pulso senoidal (SPWM), onde a largura dos pulsos que mantém os transistores acionados não é constante, estratégia para deslocar o conteúdo harmônico para altas frequências (FERRONI, 2018). Essa técnica consiste em usar uma senoide como sinal modulante na geração do PWM juntamente com uma onda triangular como portadora conforme a Figura 3(a) (RAMOS, 2018). A Figura 6 ilustra o seu funcionamento para o chaveamento do transistor S_1 da Figura 4 por exemplo.

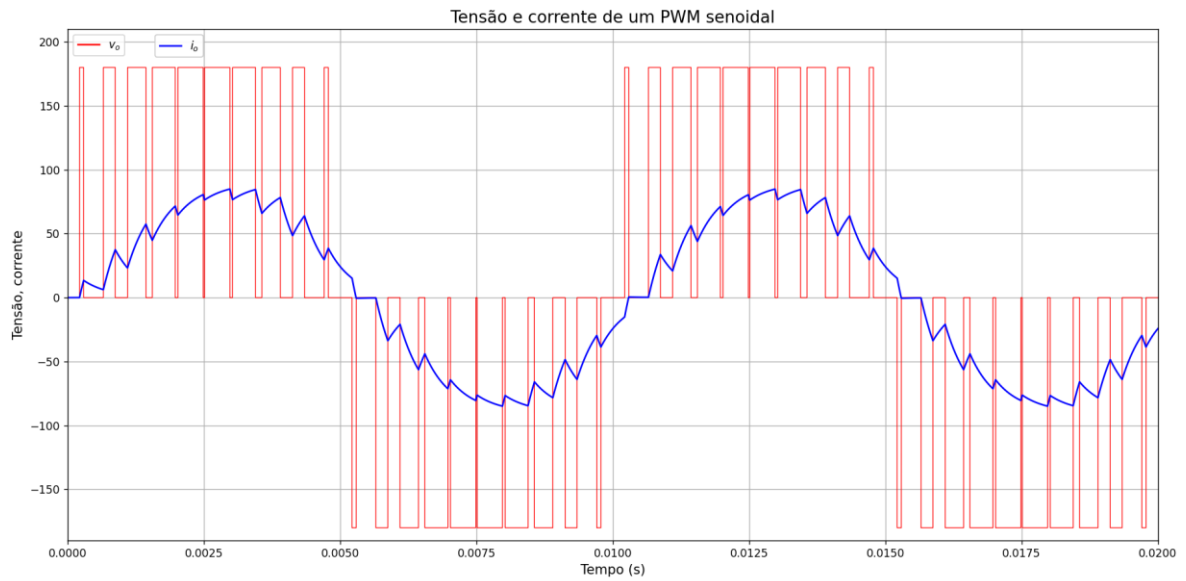
Figura 6 – Forma de onda: a) modulação do SPWM, b) sinal modulado do SPWM



Fonte: O autor

Para um inversor monofásico do tipo ponte completa é comumente utilizado o PWM a 3 níveis, que consiste em usar duas senoides defasadas de 180° entre si como sinal modulante de cada braço (FERRONI, 2018). Esse método contribui para deslocar as harmônicas para frequências mais altas ainda para uma melhor filtragem (TREVISIO, 2006). A Figura 7 exibe a forma de onda de tensão e corrente de saída do inversor para uma onda portadora a $1kHz$ e onda modulante de $100Hz$.

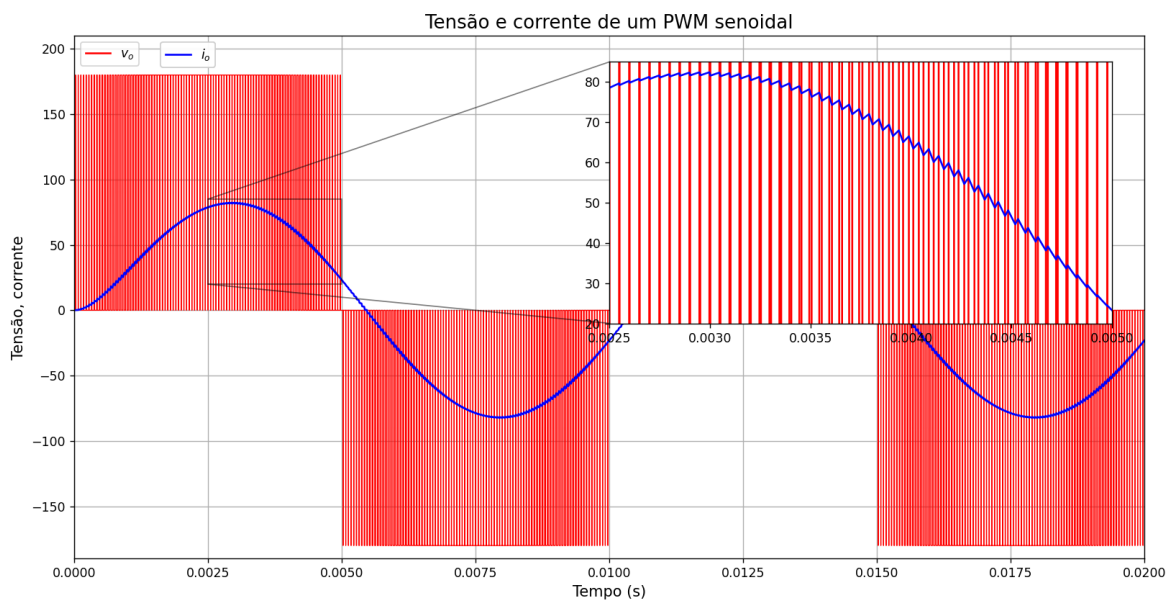
Figura 7 - SPWM com baixa frequência de chaveamento



Fonte: O autor

É possível perceber a deformidade do sinal da corrente, que não é totalmente senoidal, ou seja, com conteúdo harmônico presente. Isso se deve a baixa frequência de chaveamento, que faz com que o indutor carregue e descarregue por mais tempo, criando um *ripple* maior (RAMOS, 2018) e (FRANCHI, 2013). A Figura 8 exibe o mesmo caso com uma frequência de chaveamento de 10 kHz, sendo perceptível a diminuição da distorção harmônica da onda.

Figura 8 - SPWM com alta frequência de chaveamento

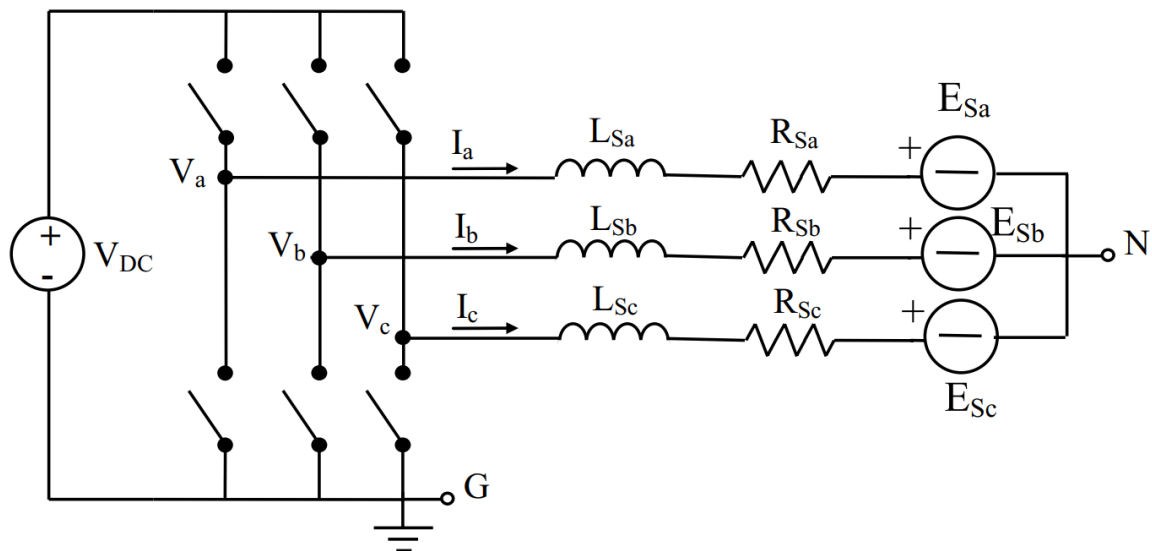


Fonte: O autor

2.4. Inversor trifásico

O inversor trifásico consiste, de forma geral, em três braços com duas chaves que operam de forma complementar entre si em cada braço. O ponto central de cada braço é conectado a uma fase da carga, que pode ser um motor trifásico por exemplo. A Figura 9 exibe o circuito do inversor trifásico.

Figura 9 - Circuito de um inversor trifásico



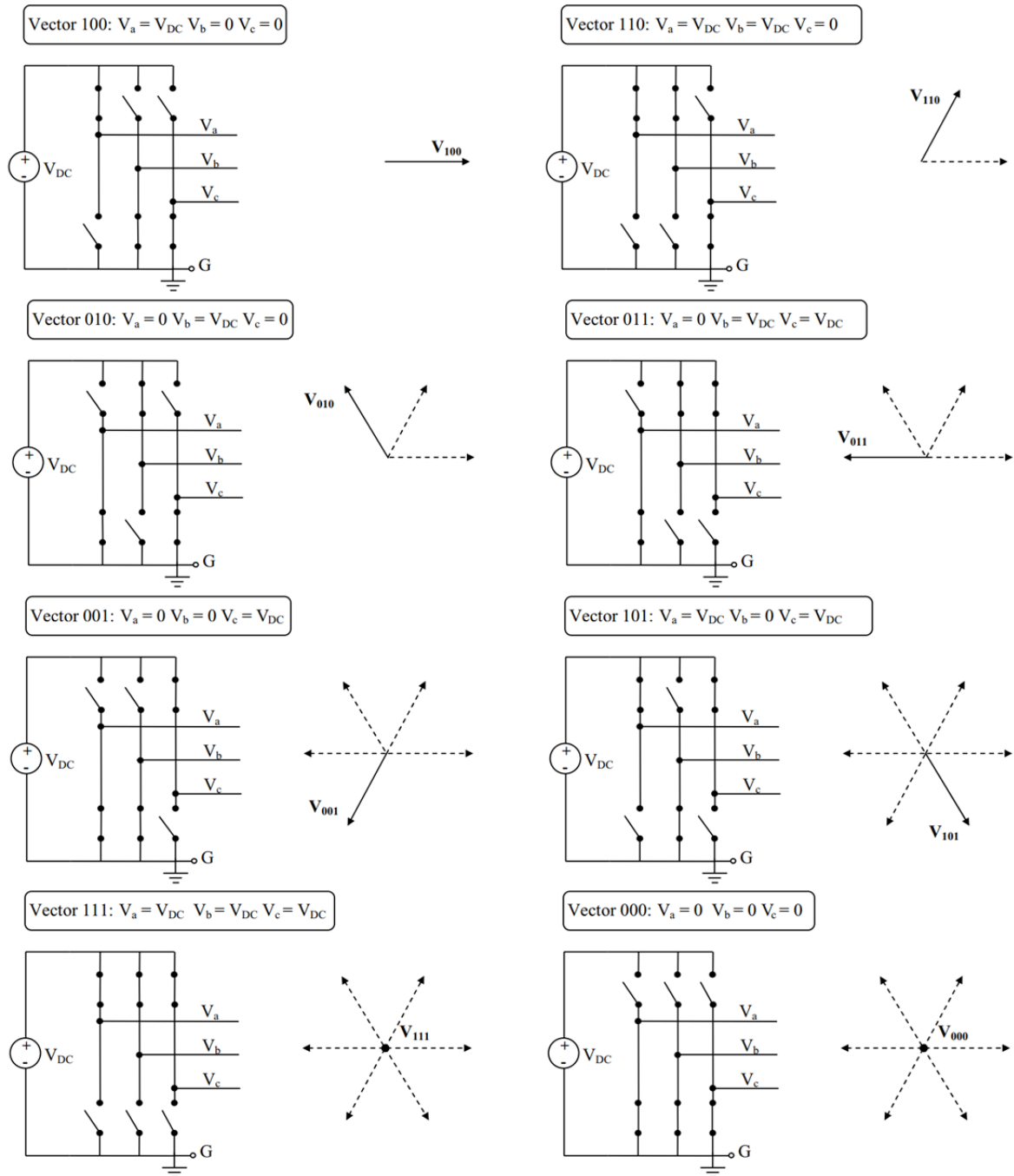
Fonte: (BUSO, 2015)

A estratégia de controle pode ser similar à que é implementada em inversores monofásicos, com três sinais modulantes, um para cada braço, sendo eles senoides defasadas de 120° entre si (DE GASPERI, 2019) ou pode ser utilizada a modulação por vetores espaciais (SVM), sendo esta segunda alternativa explorada neste trabalho.

2.5. Modulação vetorial espacial

O SVM é uma estratégia de modulação em que os valores de tensão possíveis para o conversor são agrupados em um diagrama vetorial de forma que cada um destes valores possua um vetor correspondente (SIMÕES FILHO, 2019). Estes vetores espaciais correspondentes que podem ser formados pelo inversor trifásico são determinados por todas as possíveis combinações de estados das chaves do inversor (BRITO, 2014), conforme mostra a Figura 10.

Figura 10 - Vetores de tensão de saída do inversor trifásico

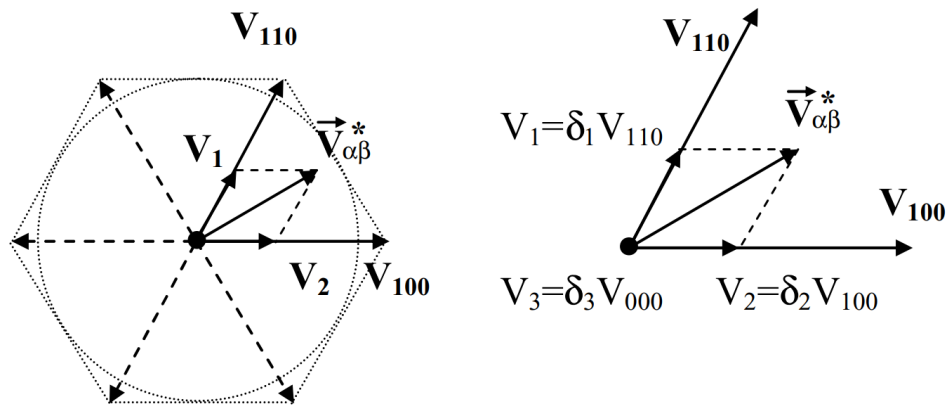


Fonte: (BUSO, 2015)

O PWM vetorial é disposto de tal maneira em que as três variáveis de tensão do inversor sejam representadas por duas variáveis, no referencial $\alpha\beta$ (SIMÕES FILHO, 2019) e (BUSO, 2015), assim é realizada a combinação de chaves do inversor com seus respectivos valores de tensão abc e calculado seu equivalente no referencial de Clarke (α e β). A implementação do SVM parte do seguinte princípio: da superposição dos vetores de saída do inversor obtém-se um vetor de tensão de saída desejado, representado no referencial $\alpha\beta$, de forma que, a média

deste sinal ao final do período de chaveamento terá sido uma tensão igual a desejada (BUSO, 2015) em cada uma das fases do inversor. Para tanto, aplica-se um dos vetores ativos (não nulos), mostrados na Figura 10, durante uma fração do período de chaveamento, outro vetor ativo durante outra fração desse período de chaveamento e na parte restante, aplica-se vetor nulo (BRITO, 2014). O procedimento pode ser explicado consultando a Figura 11. Em qualquer período de modulação, dados os componentes α e β do vetor de referência de tensão $V_{\alpha\beta}^*$, deve-se: (i) localizar os dois vetores de saída do inversor mais próximos, ou seja, o setor hexagonal onde $V_{\alpha\beta}^*$ está localizado, (ii) determinar a amplitude de V_1 e V_2 e (iii) calcular os valores δ_1 , δ_2 e δ_3 usando (7) e (8). A maneira mais simples de realizar esses cálculos é através de um microcontrolador (BUSO, 2015).

Figura 11 - Geração do vetor de referência de tensão por superposição de vetores de saída do inversor



Fonte: (BUSO, 2015)

O comprimento de cada projeção, V_1 e V_2 , determina a fração δ do período de modulação que será ocupado por cada vetor de saída, conforme (7) para quaisquer vetores de saída não nulos do inversor.

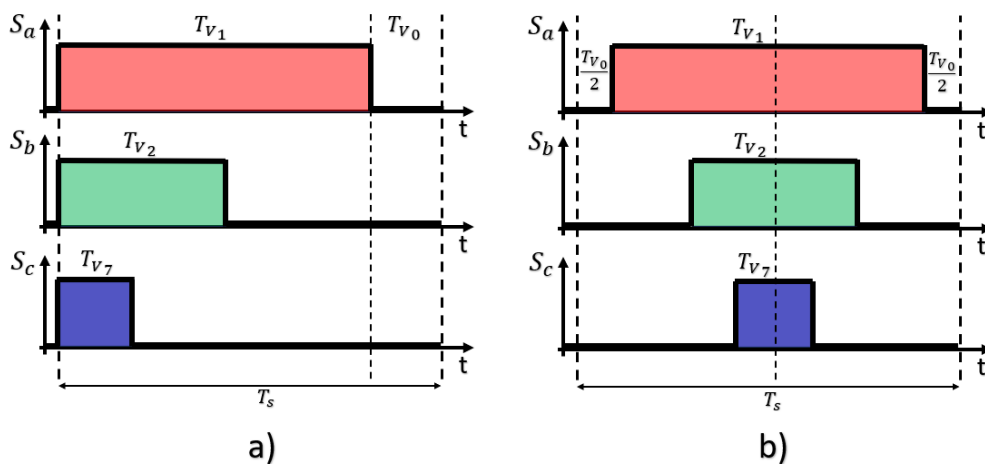
$$\delta_1 = \frac{|V_1|}{|V_{110}|}, \quad \delta_2 = \frac{|V_2|}{|V_{100}|} \quad (7)$$

A aplicação do vetor de tensão nulo para uma fração δ_3 do período de modulação é calculada com base na condição descrita em (8).

$$\delta_1 + \delta_2 + \delta_3 = 1 \quad (8)$$

Que simplesmente expressa o fato de que o período de modulação deve ser totalmente ocupado por vetores de tensão de saída. Algo importante a destacar é que o vetor zero pode ser V_{111} ou V_{000} e a ordem de aplicação dos vetores de saída do inversor é arbitrária e pode ser usada com um grau de liberdade na implementação do SVM (BUSO, 2015). O PWM trifásico obtido pela utilização da técnica SVM, é formado pelo padrão de chaveamento da Figura 12. A Figura 12(a) mostra um chaveamento não simétrico e a Figura 12(b) mostra um chaveamento simétrico.

Figura 12 - Padrão de chaveamento do SVM

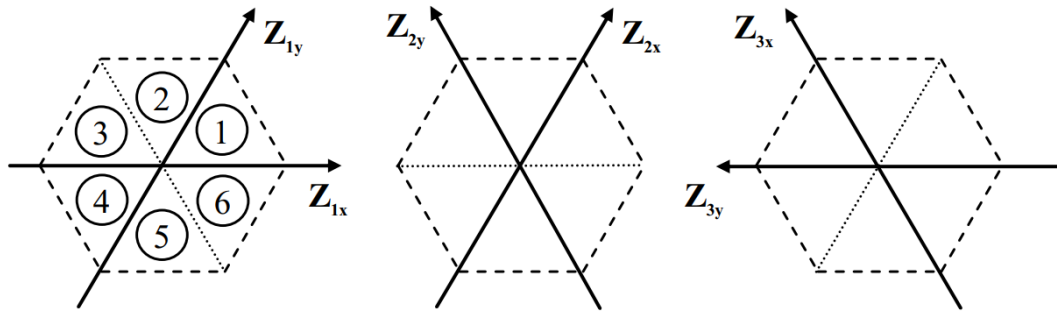


Fonte: O autor

2.5.1. Algoritmo de implementação do SVM

O primeiro passo na implementação da modulação vetorial espacial é a identificação do setor do hexágono onde o vetor de referência está localizado. Isso pode ser feito implementando uma mudança de base do referencial $\alpha\beta$ para um novo conjunto de três referenciais diferentes, onde cada quadro refere-se a um par particular de setores hexagonais (BUSO, 2015), conforme mostra a Figura 13, a esquerda o primeiro referencial, no centro o segundo e a direita o terceiro.

Figura 13 - Referenciais dos vetores espaciais



Fonte: (BUSO, 2015)

Esse método requer a projeção do vetor de referência de tensão de saída $V_{\alpha\beta}^*$ do inversor em cada um dos três pares de referenciais do hexágono (Z_{1x} e Z_{1y} , Z_{2x} e Z_{2y} e Z_{3x} e Z_{3y}) conforme (9).

$$V_{\alpha\beta}^* = Z_{ix}^* + Z_{iy}^* \quad (9)$$

Para o primeiro setor tem-se em (10):

$$V_{\alpha} + jV_{\beta} = Z_{1x} \cos(0^\circ) + jZ_{1x} \sin(0^\circ) + Z_{1y} \cos(60^\circ) + jZ_{1y} \sin(60^\circ) \quad (10)$$

Separando as partes reais e imaginárias, chega-se em (11):

$$\begin{cases} V_{\alpha} = Z_{1x} + \frac{1}{2}Z_{1y} \\ V_{\beta} = 0 + \frac{\sqrt{3}}{2}Z_{1y} \end{cases} \quad (11)$$

Na forma matricial em (12):

$$\begin{bmatrix} V_{\alpha} \\ V_{\beta} \end{bmatrix} = \begin{bmatrix} 1 & 1/2 \\ 0 & \sqrt{3}/2 \end{bmatrix} \cdot \begin{bmatrix} Z_{1x} \\ Z_{1y} \end{bmatrix} \quad (12)$$

Como o objetivo é obter Z_{1x} e Z_{1y} , realizando a inversa da matriz para isolar esses termos, tem-se em (13):

$$\begin{vmatrix} Z_{1x} \\ Z_{1y} \end{vmatrix} = \begin{vmatrix} 1 & -1/\sqrt{3} \\ 0 & 2/\sqrt{3} \end{vmatrix} \cdot \begin{vmatrix} V_\alpha \\ V_\beta \end{vmatrix} \quad (13)$$

Realizando os mesmos passos para os outros dois setores obtém-se as seguintes relações em (14):

$$\begin{vmatrix} Z_{2x} \\ Z_{2y} \end{vmatrix} = \begin{vmatrix} 1 & 1/\sqrt{3} \\ -1 & 1/\sqrt{3} \end{vmatrix} \cdot \begin{vmatrix} V_\alpha \\ V_\beta \end{vmatrix} \quad \begin{vmatrix} Z_{3x} \\ Z_{3y} \end{vmatrix} = \begin{vmatrix} 0 & 2/\sqrt{3} \\ -1 & -1/\sqrt{3} \end{vmatrix} \cdot \begin{vmatrix} V_\alpha \\ V_\beta \end{vmatrix} \quad (14)$$

A fim de minimizar os cálculos necessários, segue o seguinte algoritmo para obter os valores de todos os eixos com base em (13) e (14):

$tmp = V_\beta/\sqrt{3}$; salvar em um registrador temporário

$$Z_{1x} = V_\alpha - tmp;$$

$$Z_{2y} = -Z_{1x};$$

$$Z_{1y} = 2tmp;$$

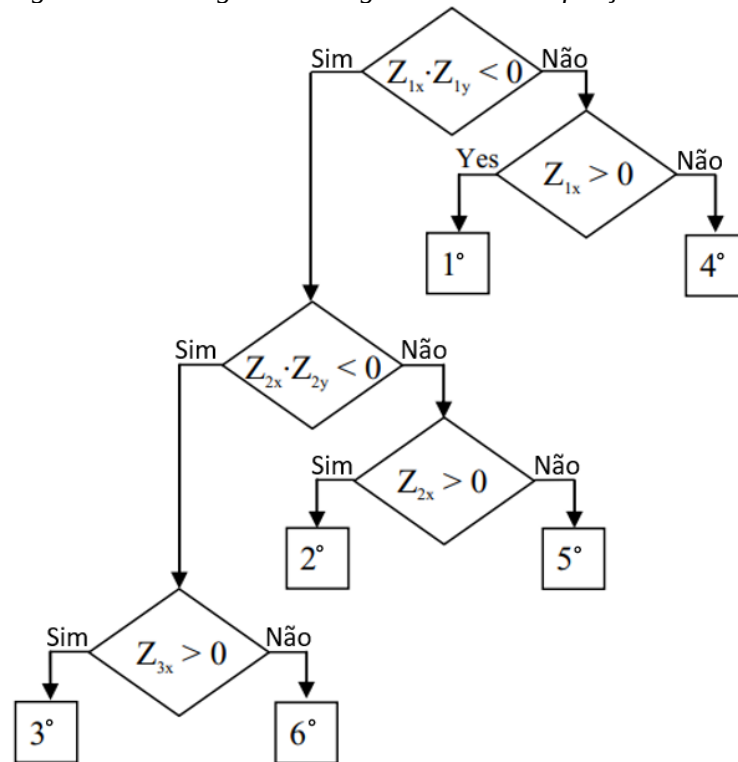
$$Z_{3x} = Z_{1y};$$

$$Z_{2x} = V_\alpha + tmp;$$

$$Z_{3y} = -Z_{2x};$$

Uma vez que as componentes Z_{ix} e Z_{iy} são conhecidas, basta determinar o setor do hexágono verificando seu sinal e aplicar os valores do período de modulação de cada chave. O fluxograma da Figura 14 descreve o procedimento de verificações de sinais que pode ser eficientemente implementada com operações lógicas no microcontrolador.

Figura 14 - Fluxograma do algoritmo de identificação do setor



Fonte: (BUSO, 2015)

Os valores de ciclo de trabalho que serão empregados na modulação de cada braço do inversor, considerando cada setor dos vetores espaciais são obtidos de acordo com os cálculos da Tabela 1.

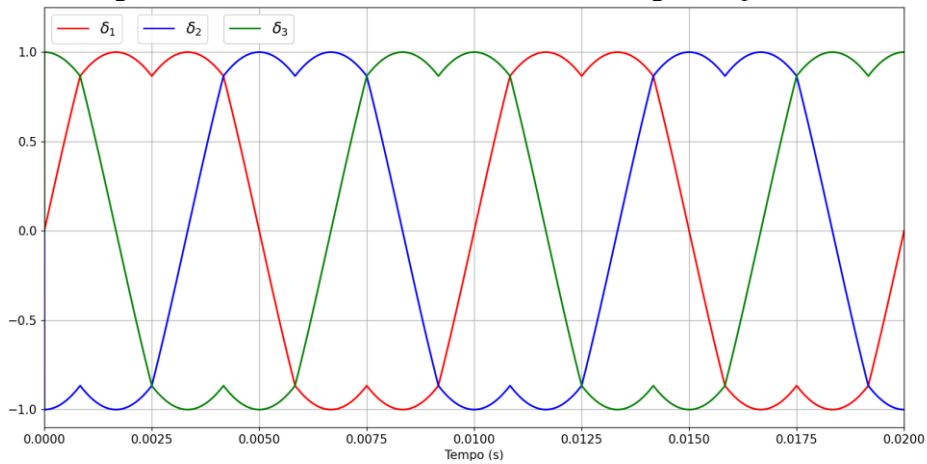
Tabela 1 - Período de modulação das chaves para cada setor dos vetores espaciais

Setor	Ciclo de trabalho		
1 ou 4	$\delta_1 = \frac{\sqrt{3}}{2}(z_{1x} + z_{1y}),$	$\delta_2 = \frac{\sqrt{3}}{2}(-z_{1x} + z_{1y}),$	$\delta_3 = \frac{\sqrt{3}}{2}(-z_{1x} - z_{1y})$
2 ou 5	$\delta_1 = \frac{\sqrt{3}}{2}(z_{2x} - z_{2y}),$	$\delta_2 = \frac{\sqrt{3}}{2}(-z_{2x} + z_{2y}),$	$\delta_3 = \frac{\sqrt{3}}{2}(-z_{2x} - z_{2y})$
3 ou 6	$\delta_1 = \frac{\sqrt{3}}{2}(-z_{3x} - z_{3y}),$	$\delta_2 = \frac{\sqrt{3}}{2}(z_{3x} + z_{3y}),$	$\delta_3 = \frac{\sqrt{3}}{2}(-z_{3x} + z_{3y})$

Fonte: Adaptado de (BUSO, 2015) e (NEACSU, 2001)

Esse tipo de modulação gera uma forma de onda característica da modulação vetorial espacial, como pode ser visto na Figura 15.

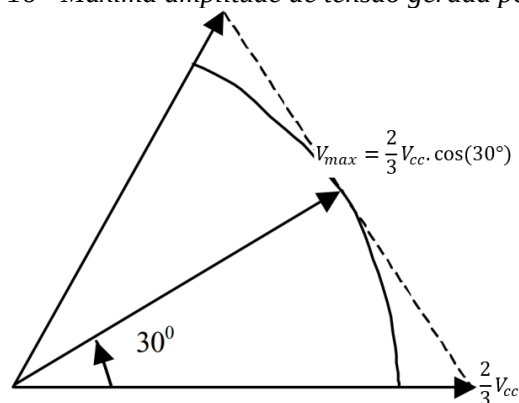
Figura 15 - Forma de onda do sinal modulante gerada pelo SVM



Fonte: O autor

Seu efeito, do ponto de vista do inversor, pode ser muito semelhante ao da utilização da estratégia PWM senoidal convencional acrescido da injeção de terceira harmônica, às vezes empregada em implementações analógicas trifásicas de PWM. Obtém-se um aumento de 15% da faixa tensão-amplitude que corresponde a uma operação linear do conversor, ou seja, à ausência de qualquer fenômeno de saturação (BUSO, 2015). Isso pode ser constatado ao calcular a amplitude máxima V_{max} do triplete de tensões que pode ser gerada em (1) para a transformação $\alpha\beta$ invariante em amplitude, que corresponde a um vetor girante com amplitude igual ao raio do círculo inscrito na Figura 11, conforme mostra a Figura 16 e (15).

Figura 16 - Máxima amplitude de tensão gerada pelo SVM



Fonte: Adaptado de (NEACSU, 2001)

$$V_{max} = \frac{2}{3} V_{cc} \frac{\sqrt{3}}{2} \Leftrightarrow V_{max} \cong 1.1547 \frac{V_{cc}}{2} \quad (15)$$

O sinal modulante da Figura 15 é um valor entre -1 e 1 e deve ser comparado com a informação da onda portadora que o timer do microcontrolador vai gerar. Para isso, é necessário realizar a normalização do sinal SVM para estar compatível com a portadora, assim, é feito o cálculo descrito em (16).

$$S_{modulante} = i_{mod} \cdot T_{timer} \cdot (1 + \delta_i) \cdot 0,5 \quad (16)$$

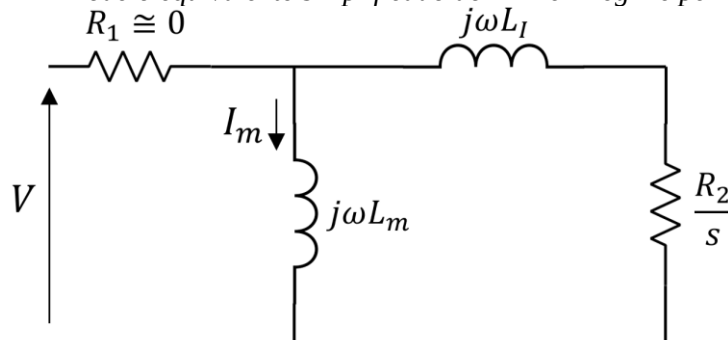
Para um contador no microcontrolador que conta até um valor T_{timer} (por exemplo um contador de 8 bits que vai até 255), o sinal modulante deve corresponder proporcionalmente ao valor desse contador. O sinal do SVM passará a ser um valor entre 0 e 1 multiplicado pelo valor do contador e um índice de modulação i_{mod} que vai controlar a amplitude máxima do valor gerado.

2.6. Controle escalar V/F do motor de indução trifásico

O princípio de controle V/F (tensão eficaz/frequência) se baseia na manutenção do fluxo de estator constante e igual ao seu valor nominal, independente da frequência de acionamento e de carga no eixo do motor (DE ARAUJO SILVA, 2000), desta forma garantindo que o torque no eixo do motor seja constante independentemente da velocidade. Desta maneira, mantém-se a capacidade de torque da máquina tornando possível o acionamento em ampla faixa de velocidades. Ao manter a relação V/F de alimentação do motor de indução trifásico (MIT), de modo que quando se altera a frequência da tensão que alimenta o motor, com objetivo de alterar sua velocidade de rotação, o controle modifica a grandeza tensão na mesma proporção (BERTOLUCCI, 2022).

O modelo simplificado de um MIT é mostrado na Figura 17.

Figura 17 - Modelo equivalente simplificado do MIT em regime permanente



Fonte: O autor

Nesse modelo a resistência do estator ($R_1 \cong 0$) e as perdas no núcleo são desprezadas, além de se incorporar em um único elemento as indutâncias de dispersão do estator (L_{ls}) e de dispersão do rotor (L'_{lr}), como indicado em (17).

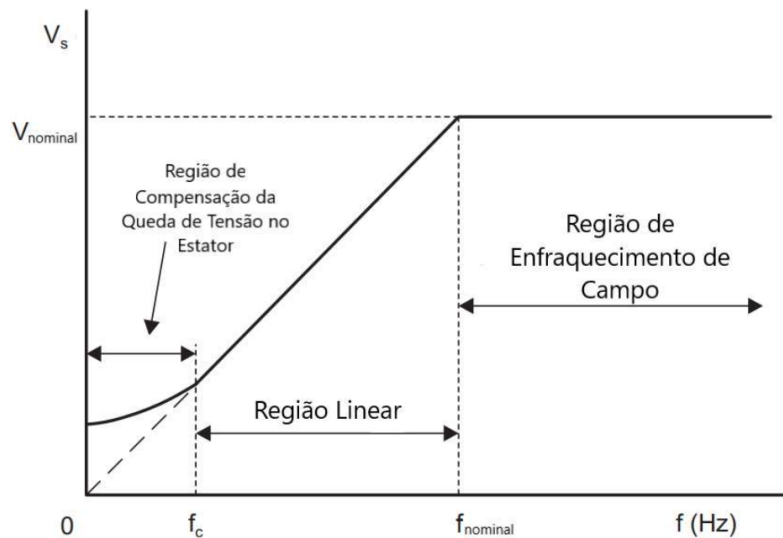
$$L_l = L_{ls} + L_{lr} \quad (17)$$

Nessas condições, a corrente de magnetização I_m , responsável por gerar o fluxo no entreferro, pode ser aproximada de acordo com (18).

$$I_m = \frac{V}{2\pi f L_m} \quad (18)$$

De (18), tem-se que se mantida constante a relação V/f , o fluxo magnético permanece constante. Contudo, em velocidades muito baixas (frequência e tensão baixas), a resistência R_1 deixa de ser desprezível, ocasionando uma queda de tensão relevante em relação a tensão sobre a reatância de magnetização, comprometendo o torque desenvolvido (BERTOLUCCI, 2022), visto que a tensão resultante sobre a indutância de magnetização diminui, diminuindo assim o fluxo no motor. Desta forma, surge a necessidade de uma compensação para esta queda de tensão em baixas frequências, que pode ser feita por meio de um valor mínimo de tensão ou de frequência. O perfil típico de um controle escalar é ilustrado na Figura 18.

Figura 18 - Perfil de tensão no estator versus frequência em modo de controle escalar

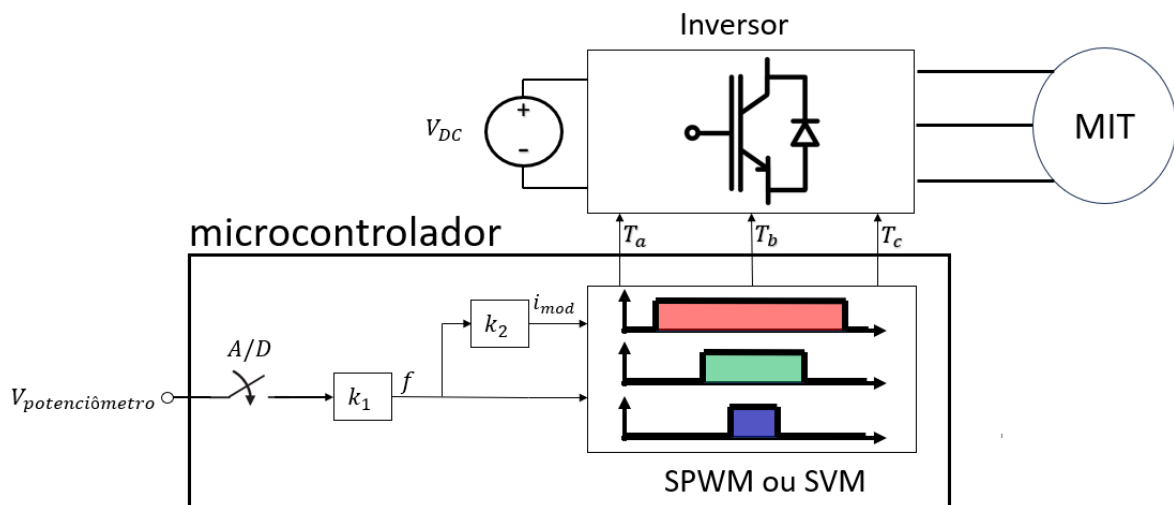


Fonte: (BERTOLUCCI, 2022)

Na região entre zero e f_c , mostrada na Figura 18, verifica-se a necessidade de compensação de tensão para compensar a queda de tensão na resistência interna do motor. Entre f_c e $f_{nominal}$, tem-se a região linear, onde é mantida constante a relação V/F. Por outro lado, na região em que a frequência é maior que $f_{nominal}$, a tensão é limitada no valor de tensão nominal do MIT e a relação V/F não pode ser mantida, criando um enfraquecimento de campo, conforme (18).

O controle escalar, em sua forma mais simples, pode ser implementado em malha aberta, ou seja, sem realimentação. O usuário entra com uma referência de frequência f e com isso é realizado o ajuste da tensão para mantê-la proporcional à frequência, esta etapa é feita modificando o índice de modulação i_{mod} do PWM trifásico, seja implementando por meio de modulação PWM senoidal ou por modulação vetorial espacial. A Figura 19 mostra um diagrama de blocos de como funciona o controle.

Figura 19 - Diagrama de blocos do controle V/F em malha aberta



Fonte: O autor

O bloco k_1 relaciona a informação obtida da conversão analógica digital para a faixa de frequência de giro do motor escolhida. O bloco k_2 é responsável por fazer o ajuste de tensão através do índice de modulação com base na frequência de referência. Com esses dados, será formado o sistema de tensões trifásico que constituem os sinais modulantes do controle PWM, que por sua vez gera os tempos de chaveamento do inversor que alimenta o motor.

Se tratando da máquina de indução, a velocidade de rotação do rotor nunca será igual à velocidade de referência devido ao escorregamento.

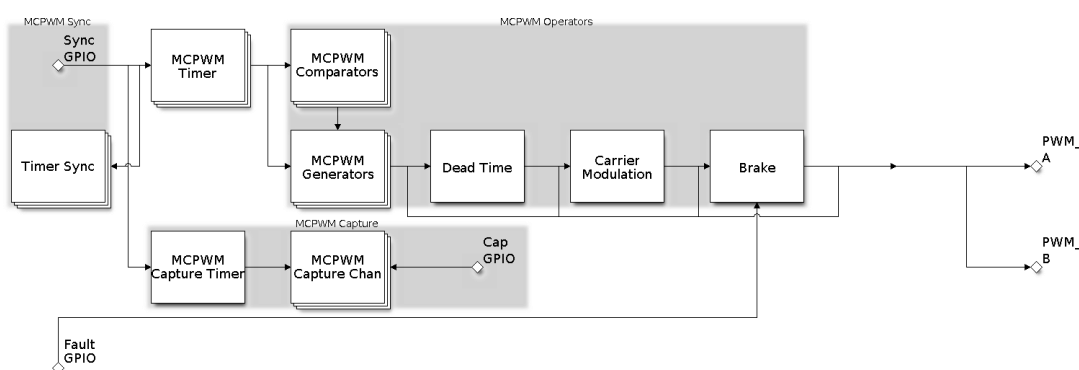
2.7. Microcontrolador ESP32

O ESP32 é um microcontrolador moderno de 32 bits desenvolvido pela Espressif Systems, é reconhecido pela sua versatilidade, oferecendo suporte a Wi-Fi, Bluetooth e outras tecnologias de conexão. Com um processador dual-core de até 240 MHz, 520 kB de RAM, 4 MB de memória flash interna e uma gama de periféricos, como UART, SPI, I2C, o ESP32 é ideal para projetos de IoT. Além disso, esse microcontrolador possui um módulo especial dedicado a aplicações de eletrônica de potência: o MCPWM. Conforme a documentação em (ESP32 TECHNICAL REFERENCE MANUAL, 2024) e (TECHNICAL DOCUMENTS | ESPRESSIF SYSTEMS, 2024).

2.7.1. Módulo MCPWM do ESP32

O periférico MCPWM é um gerador PWM versátil, que contém vários submódulos para torná-lo um elemento chave em aplicações de eletrônica de potência, como controle de motor, (MOTOR CONTROL PULSE WIDTH MODULATOR (MCPWM) - ESP32, 2024). Seus principais submódulos são descritos na Figura 20.

Figura 20 - Visão geral do MCPWM



Fonte: (MOTOR CONTROL PULSE WIDTH MODULATOR (MCPWM) - ESP32, 2024)

Normalmente, o periférico MCPWM pode ser usado nos seguintes cenários:

- Controle digital de motor, por exemplo, motor DC com/sem escovas e servo motor;
- Conversão de energia baseada em chaveamento;
- DAC de potência, onde o ciclo de trabalho é equivalente a um valor analógico do DAC;
- Calcular a largura do pulso externo e convertê-la em outros valores analógicos, como velocidade e distância;
- Gerar sinais PWM de vetor espacial (SVPWM) para controle orientado a campo, do inglês *Field Oriented Control* (FOC).

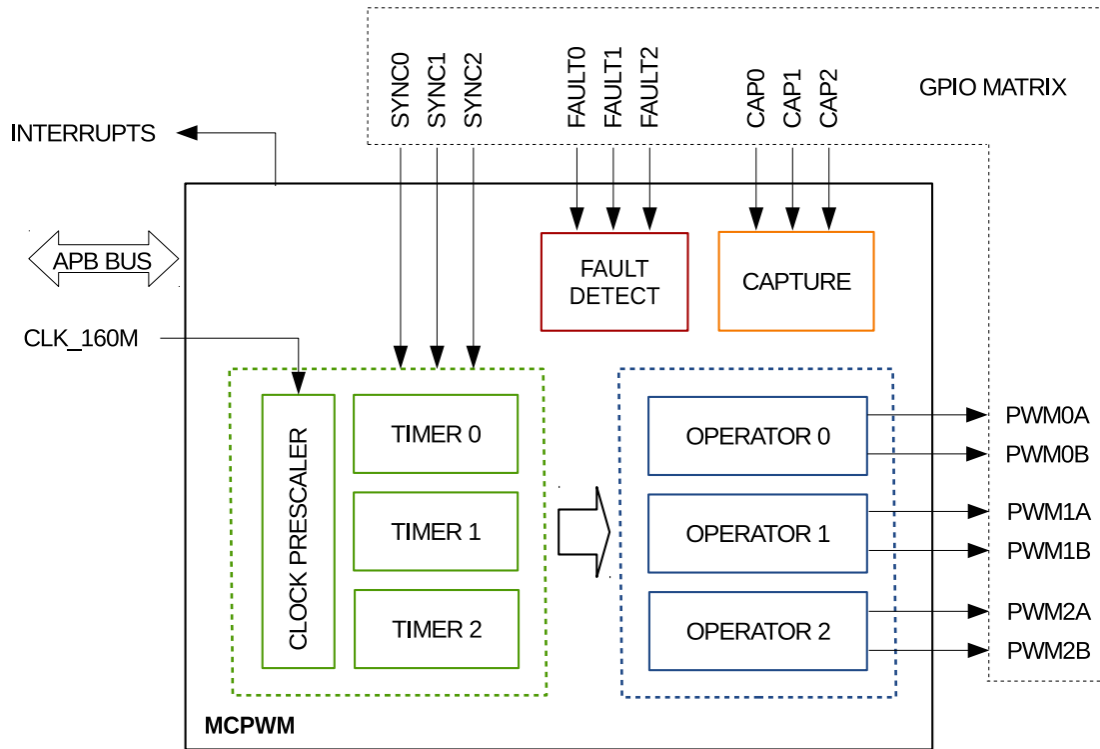
Seus principais submódulos são (MOTOR CONTROL PULSE WIDTH MODULATOR (MCPWM) - ESP32, 2024):

- **MCPWM Timer:** A base de tempo do sinal PWM final. Ele também determina o tempo de evento de outros submódulos.
- **Operador MCPWM:** O módulo-chave que é responsável por gerar as formas de onda PWM. Ele consiste em outros submódulos, como comparador, gerador PWM, tempo morto e modulador de portadora.
- **Comparador MCPWM:** O módulo compare pega o valor de contagem de base de tempo como entrada e o compara continuamente ao valor limite configurado. Quando o timer for igual a qualquer um dos valores limite, um evento compare será gerado e o gerador MCPWM pode atualizar seu nível de acordo.
- **Gerador MCPWM:** Um gerador MCPWM pode gerar um par de ondas PWM, complementar ou independentemente, com base em vários eventos acionados por outros submódulos, como o Temporizador MCPWM e o Comparador MCPWM.
- **MCPWM Fault:** O módulo de falha é usado para detectar a condição de falha de fora, principalmente por meio da matriz GPIO. Uma vez que o sinal de falha esteja ativo, o MCPWM Operator forçará todos os geradores a um estado predefinido para proteger o sistema contra danos.
- **MCPWM Sync:** O módulo de sincronização é usado para sincronizar os temporizadores MCPWM, de modo que os sinais PWM finais gerados por diferentes geradores MCPWM possam ter uma diferença de fase fixa. O sinal de sincronização pode ser roteado da matriz GPIO ou de um evento MCPWM Timer.
- **Tempo morto:** este submódulo é usado para inserir atraso extra nas bordas PWM existentes geradas nas etapas anteriores.
- **Modulação de portadora:** O submódulo de portadora pode modular um sinal de portadora de alta frequência em formas de onda PWM pelo gerador e submódulos de tempo morto. Essa capacidade é obrigatória para controlar os elementos de comutação de energia.
- **Freio:** O operador MCPWM pode definir como frear os geradores quando uma falha específica for detectada. Você pode desligar a saída PWM imediatamente ou regular a saída PWM ciclo por ciclo, dependendo de quão crítica a falha é.
- **Captura MCPWM:** Este é um submódulo autônomo que pode funcionar mesmo sem os operadores MCPWM acima. A captura consiste em um temporizador dedicado e vários canais independentes, com cada canal conectado ao GPIO. Um pulso no GPIO aciona o temporizador de captura para armazenar o valor da contagem da base de tempo e, em seguida, notificá-lo por uma interrupção. Usando esse recurso, você pode medir uma largura de pulso com precisão. Além disso, o temporizador de captura também pode ser sincronizado pelo submódulo MCPWM Sync.

O ESP32 tem duas unidades MCPWM que podem ser usadas para controlar diferentes tipos de motores (PWM0 a partir do registrador `0x3FF5E00` e PWM1 a partir do registrador `0x3FF6C00`). Cada unidade tem três pares de saídas PWM, o que facilita a programação para

controle de motores trifásicos. A Figura 21 exibe um diagrama de blocos detalhado de uma unidade MCPWM, (MCPWM - ESP32, 2024).

Figura 21 - Diagrama de blocos de uma unidade MCPWM



Fonte: (MCPWM - ESP32, 2024)

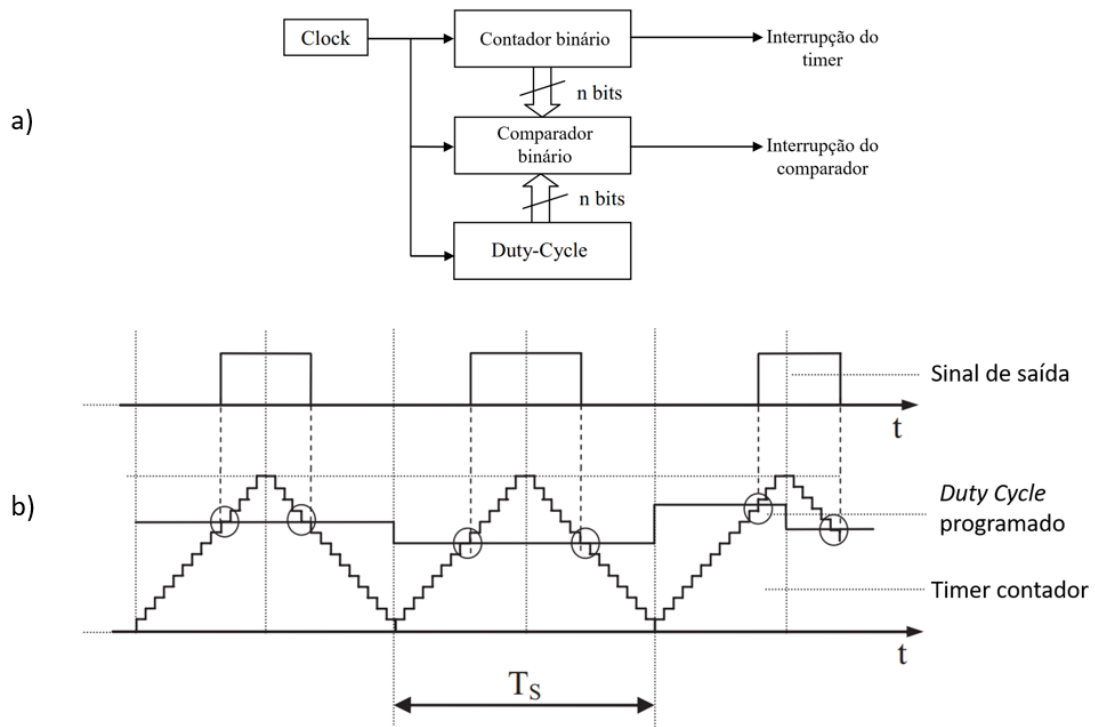
Cada par A/B de PWM pode ser cronometrado por qualquer um dos três temporizadores Timer 0, 1 e 2. O mesmo temporizador pode ser usado para cronometrar mais de um par de saídas PWM (como é o caso deste trabalho). Cada unidade também é capaz de coletar entradas como detectar sobrecorrente ou sobretensão do motor, bem como obter feedback com, por exemplo, uma posição do rotor, (MCPWM – ESP32, 2024) o que é de grande utilidade para controle do motor em malha fechada.

2.7.2. PWM digital com MCPWM do ESP32

Os princípios básicos descritos na seção 2.2 se aplicam também à implementação digital do modulador PWM. Na implementação mais direta, também conhecida como “PWM uniformemente amostrado”, cada bloco analógico é substituído por um digital. A função do comparador analógico é substituída por um comparador digital, o gerador de portadora é substituído por um contador binário, e assim por diante (BUSO, 2015). A Figura 22 mostra a organização típica de hardware de um PWM digital, do tipo que podemos encontrar dentro de

vários microcontroladores e processadores de sinais digitais, seja como uma unidade periférica dedicada ou como uma função programável especial do temporizador de propósito geral.

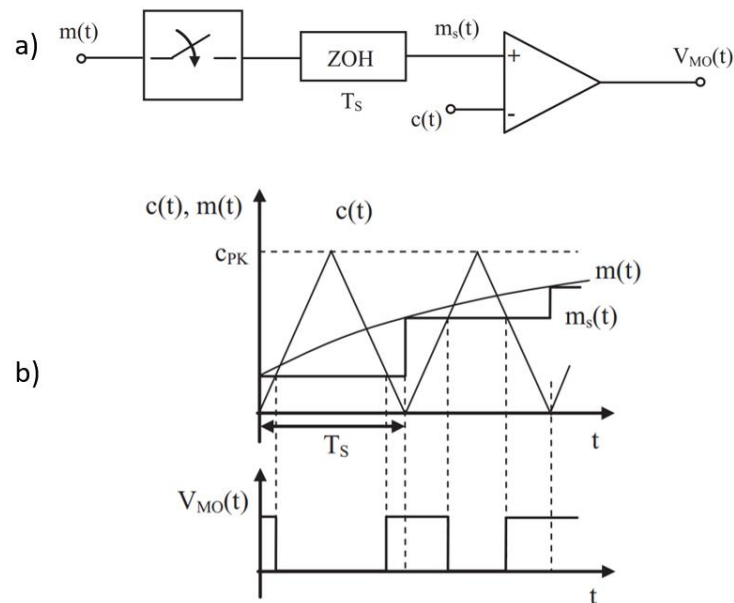
Figura 22 - PWM digital: a) organização, b) exemplo de um PWM digital



Fonte: (BUSO, 2015)

O princípio de funcionamento de um PWM digital é descrito a seguir: o contador é incrementado a cada pulso de *clock*; Sempre que o valor do contador binário for igual registrador de comparação, relacionado ao ciclo de trabalho (*duty cycle*) programado, o comparador binário dispara uma interrupção para o microprocessador e, ao mesmo tempo, define o sinal de saída como baixo. Um ponto importante do PWM digital é o atraso de resposta dinâmica do sinal modulante, onde sua atualização é realizada apenas no início de cada período de modulação (BUSO, 2015). Este efeito é modelado como “amostrar e manter o efeito”, do inglês *sample and hold*. A Figura 23 exhibe seu funcionamento.

Figura 23 - PWM digital uniformemente amostrado: a) diagrama de blocos geral, b) modulação com portadora triangular



Fonte: (BUSO, 2015)

O sinal modulante $m(t)$ é retido em $m_s(t)$ no início de cada período T_s , perdendo assim sua informação instantânea ao longo de um período de chaveamento T_s , gerando assim um sinal PWM $V_{MO}(t)$ ligeiramente diferente daquele obtido pela modulação PWM analógica. Esta perda de informação pode ser amenizada diminuindo o período de amostragem.

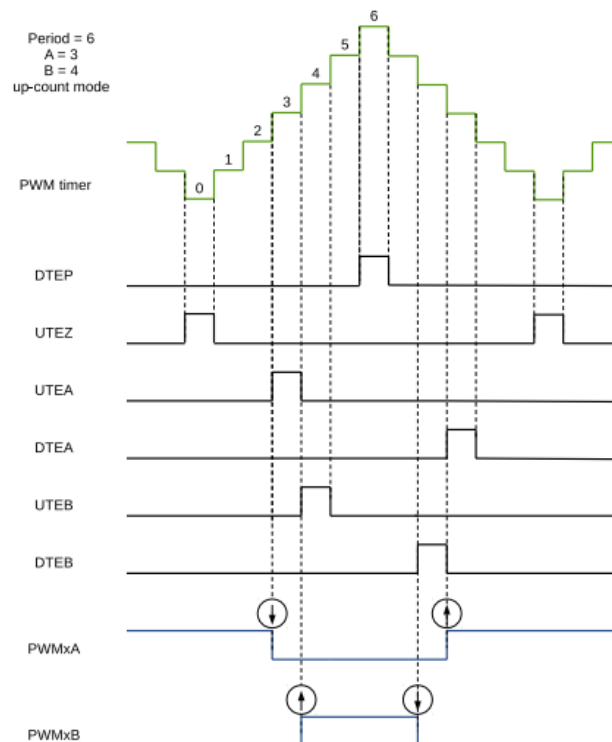
O microcontrolador ESP32 possui no seu módulo MCPWM configurações de timer especiais para aplicações de geração de onda portadora e de eventos de temporização para o comparador. O submódulo *Timer* habilita o dispositivo para realizar a contagem do *timer* em crescente, decrescente e crescente-decrescente (portadora triangular). Estas configurações são realizadas através dos registradores `PWM_CLK_CFG_REG`, `PWM_TIMERx_CFG0_REG`, `PWM_TIMERx_CFG1_REG` e `PWM_TIMERx_STATUS_REG` (ESP32 TECHNICAL REFERENCE MANUAL, 2024).

O submódulo gerador (MCPWM generators) configura importantes eventos de temporização. Os eventos são então convertidos em ações específicas para gerar as formas de ondas desejadas nas saídas `PWMxA` e `PWMxB` de cada operador, conforme Figura 21. Os registradores para essas configurações são `PWM_GENx_STMP_CFG_REG` e `PWM_GENx_TSTMP_A_REG`, `PWM_GENx_TSTMP_B_REG` (ESP32 TECHNICAL REFERENCE MANUAL, 2024). Alguns dos principais eventos são listados abaixo, de acordo com (ESP32 TECHNICAL REFERENCE MANUAL, 2024):

- UTEA: o timer PWM está em contagem crescente e seu valor é igual ao registrador A;
- UTEB: o timer PWM está em contagem crescente e seu valor é igual ao registrador B;
- DTEA: o timer PWM está em contagem regressiva e seu valor é igual ao registrador A;
- DTEB: o timer PWM está em contagem regressiva e seu valor é igual ao registrador B;
- DTEP: o timer PWM está em contagem decrescente e seu valor é igual ao valor do registrador de período;
- DTEZ: o timer PWM está em contagem decrescente e seu valor é igual a zero;
- UTEP: o timer PWM está em contagem crescente e seu valor é igual ao valor do registrador de período;
- UTEZ: o timer PWM está em contagem crescente e seu valor é igual a zero.

O submódulo gerador de PWM também controla o comportamento das saídas $PWMxA$ e $PWMxB$ a partir dos registradores $PWM_GENx_A_REG$ e $PWM_GENx_B_REG$ (ESP32 TECHNICAL REFERENCE MANUAL, 2024). A Figura 24 exibe o modo de funcionamento de um tipo de configuração PWM do ESP32.

Figura 24 - PWM digital do módulo MCPWM do ESP32



Fonte: (ESP32 TECHNICAL REFERENCE MANUAL, 2024)

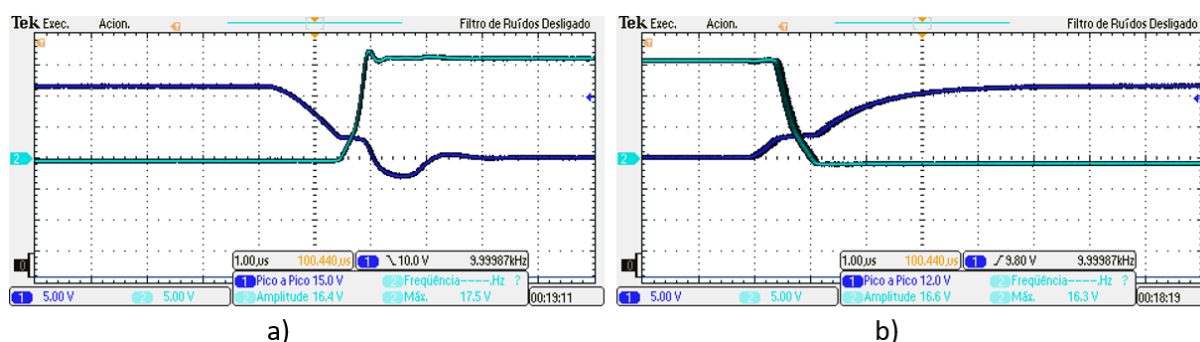
Neste caso, o microcontrolador está programado para criar dois eventos quando o timer estiver incrementando (UTEA e UTEB) e outros dois eventos ocorrem quando o timer estiver

decrementando (DTEA e DTEB). Nos eventos de incremento, o sinal de saída do PWMxA vai para nível baixo e PWMxB para nível alto, enquanto nos eventos de decremento ocorre o inverso.

2.8. Tempo morto

Teoricamente os transistores deveriam responder imediatamente à variação do sinal de acionamento, trocando o estado de condução para bloqueio e vice-versa. Contudo, na realidade há um tempo entre o acionamento e a efetiva condução total de um transistor, assim como existe um tempo entre o comando para desligar o transistor e a efetiva entrada em bloqueio. A Figura 25 exibe o momento em que um transistor é chaveado. Em azul escuro tem-se o sinal PWM e em azul claro, a tensão no transistor.

Figura 25 - Chaveamento de um transistor: a) para condução, b) para corte



Fonte: O autor

É possível perceber que são necessários alguns de microssegundos até que o dispositivo entre em condução ou em corte. Na situação do inversor, onde existem dois transistores em série no mesmo braço, esse tempo de mudança fará com que todos os semicondutores estejam ligados simultaneamente, ocasionando rápidos curtos-circuitos. A efeito de longo prazo, esses pequenos pulsos de curto-circuito podem causar o superaquecimento ou queima dos transistores (FERRONI, 2018).

Uma maneira de contornar esse problema é garantir que os dois conjuntos de transistores não operem de maneira simultânea no momento do chaveamento. Para isso, é necessário considerar um pequeno atraso no sinal de acionamento de um transistor (entrada em condução), garantindo que o desligamento do seu complementar ocorra antes da condução do outro transistor. Esta característica é denominada tempo morto.

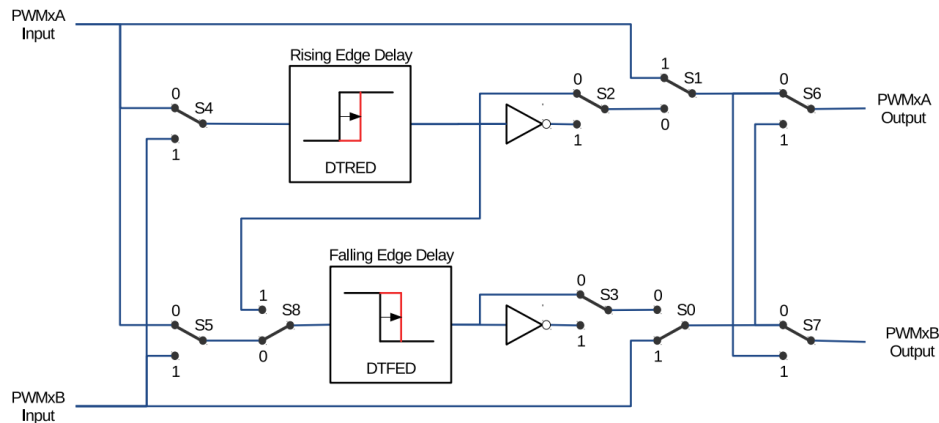
2.8.1. Tempo morto com MCPWM do ESP32

O periférico MCPWM do ESP32 possibilita a programação de tempo morto em sinais PWM com seu submódulo de tempo morto (*Dead Time*). Sua configuração é feita pelos registradores *PWM_DT_x_CFG_REG*, *PWM_DT_x_FED_CFG_REG* e *PWM_DT_x_RED_CFG_REG*. As principais funções desse submódulo são as seguintes:

- Gerar pares de sinais (PWMxA e PWMxB) com tempo morto a partir de uma única entrada PWMxA;
- Criar um tempo morto adicionando atraso às bordas do sinal:
 - Atraso de borda ascendente – *rising edge delay* (RED)
 - Atraso de borda descendente – *falling edge delay* (FED)
- Configura os pares de sinais a serem:
 - Ativo alto complementar – *active high complementary* (AHC)
 - Ativo baixo complementar – *active low complementary* (ALC)
 - Ativo alto – *active high* (AH)
 - Ativo baixo – *active low* (AL)

As opções para configurar o submódulo de tempo morto são mostradas na Figura 26.

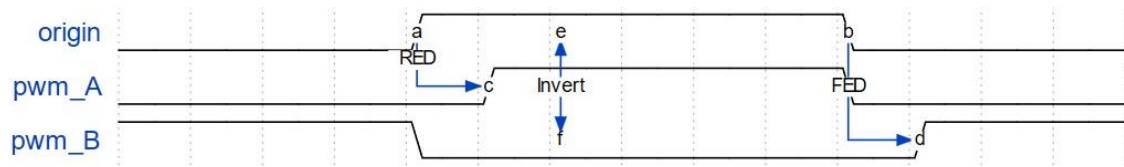
Figura 26 - Opções para configurar o submódulo do gerador de tempo morto do ESP32



Fonte: (ESP32 TECHNICAL REFERENCE MANUAL, 2024)

A configuração mais comum para inversores com dois transistores no mesmo braço é a ativo alto complementar, exibido na Figura 27.

Figura 27 - Tempo morto ativo alto complementar



Fonte: (MOTOR CONTROL PULSE WIDTH MODULATOR (MCPWM) - ESP32, 2024)

2.9. IDE e framework de programação

Neste trabalho foi utilizada *Integrated Development Environment* (IDE) VSCode com a extensão *Espressif IoT Development Framework* (ESP-IDF) em linguagem C para programação do código do projeto. O ESP-IDF fornece vários tipos de interfaces de programação, como funções C, estruturas, enumerações, definições de tipo e macros de pré-processador declaradas em arquivos de cabeçalho públicos de componentes ESP-IDF (API CONVENTIONS - ESP32, 2024).

3. MATERIAIS E MÉTODOS

3.1. Materiais

Para esse trabalho, foi utilizado os seguintes materiais:

- Microcontrolador ESP32;
- Protoboard;
- 5 Botões;
- Display LCD 16x2;
- Jumpers;
- Placa inversora com ponte H trifásica de 10kW;
- Fonte CC;
- Osciloscópio;
- Notebook para programação do microcontrolador;
- Motor de indução trifásico.

Os dados de placa do motor utilizado são exibidos na Tabela 2.

<i>Tabela 2 - Dados do motor utilizado</i>	
Dado	Valor
Potência Nominal	0,55 CV
Tensão Nominal	220/380 V
Rendimento	78,3%
FP	0,82
Rotação Nominal	1730 RPM

3.2. Método

A metodologia aplicada se deu da seguinte maneira:

- 1) Em uma primeira etapa, o projeto iniciou-se pesquisando a respeito do funcionamento de inversores monofásicos e inversores trifásicos, como chaveá-los com transistores e PWM para gerar formas de onda alternadas a partir de uma fonte CC, seguindo com o estudo do PWM senoidal para diminuir a distorção harmônica e melhorar a forma de onda gerada para o motor de indução.
- 2) O projeto deu continuidade com os estudos a respeito do microcontrolador utilizado: ESP32, com leitura da documentação técnica buscando entender o periférico MCPWM para o propósito do trabalho. Essa etapa foi importante, pois esse periférico é o motivo principal para aplicar modulação de inversores trifásicos no microcontrolador ESP32. Configurações do timer, comparadores, geradores, modo de saída do PWM, tempo morto, interrupções do timer, conversão analógica digital etc.

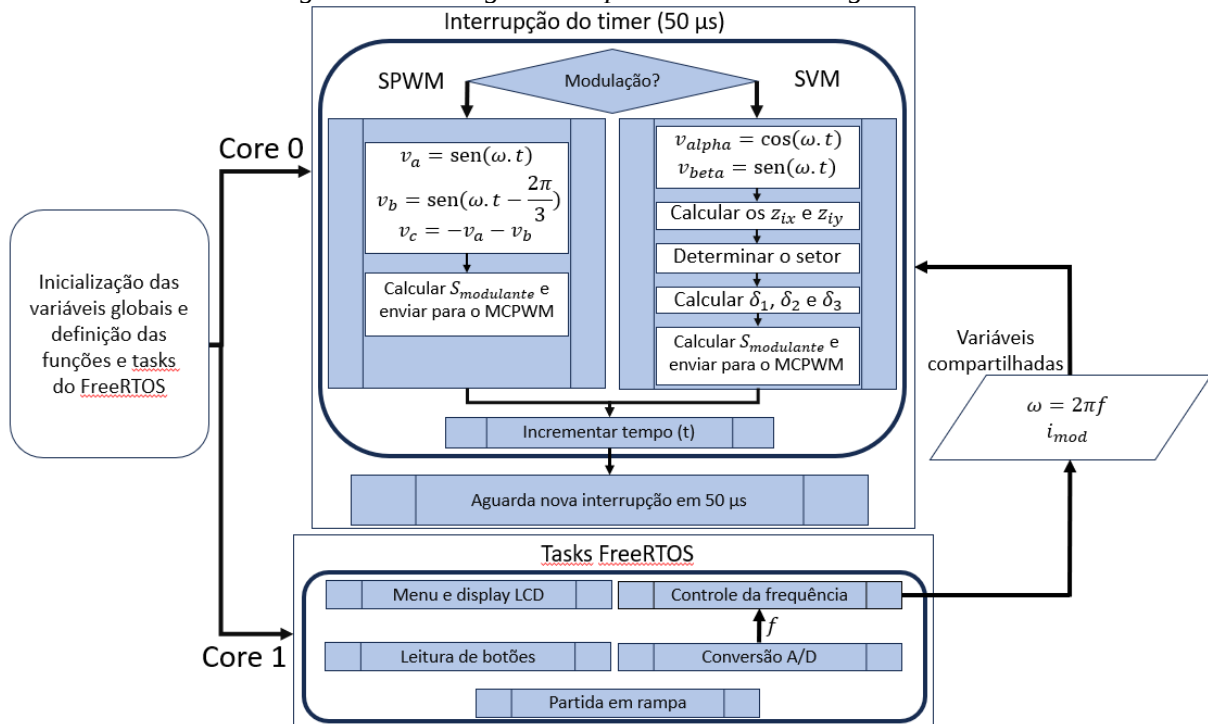
- 3) Após esses estudos, foi dado início na programação do microcontrolador para gerar o SPWM. Boa parte da configuração do periférico MCPWM foi feita com as funções prontas fornecidas pela ESPRESSIF em (MOTOR CONTROL PULSE WIDTH MODULATOR (MCPWM) - ESP32, 2024) e algumas configurações foram feitas acessando direto o endereço de memória dos registradores do ESP32. Nessa etapa também foram capturadas as formas de onda do PWM gerado pelo microcontrolador.
- 4) Com os conceitos essenciais sobre modulação de inversores trifásicos, foi dado seguimento no trabalho com a modulação vetorial espacial, primeiramente pesquisando sua teoria e aplicando a implementação no microcontrolador. Um conteúdo didático utilizado foi (Space Vector PWM Intro, 2017).
- 5) Com essa etapa pronta, foi criada uma interface para configurar os parâmetros do inversor. Um menu no display LCD, usando a protoboard com o ESP32, botões para navegação no menu e potenciômetro para modificação da frequência da senoide gerada pelo microcontrolador.
- 6) Ao fim dessa etapa, foi possível testar as modulações PWM na placa inversora, conectada a fonte CC e ao motor de indução trifásico.
- 7) Após o sucesso da etapa anterior, com o controle de giro do motor, foi possível coletar mais dados, como corrente elétrica e tensão do motor.

3.2.1. Fluxograma do algoritmo

O código é separado em tarefas (*tasks*) por meio do *freeRTOS*, que faz a execução de diversas tarefas “simultaneamente” a partir de um sofisticado sistema de timer e interrupções (FreeRTOS Documentation - FreeRTOS™, 2024). Cada tarefa criada é responsável por uma função diferente como leitura analógica, leitura dos botões, controle da frequência, partida suave etc. Essas tarefas são chamadas e executadas em períodos de tempos definidos.

O processamento das tarefas também é dividido entre os dois núcleos do ESP32. Um deles é dedicado para a geração do sinal de referência (sinal senoidal proveniente da modulação SPWM ou SVM) através de uma interrupção por timer a cada 50 μ s. Essa tarefa demanda muito tempo do núcleo, devido ao cálculo das senoides e é realizada em um período muito curto de tempo, por isso esse processador tem dedicação exclusiva para ela. O outro núcleo de processamento recebe todas as outras tarefas que aparecem no código, uma vez que elas não demandam tanto tempo de processamento em um curto período. A Figura 28 exibe o fluxograma de como o código está estruturado.

Figura 28 - Fluxograma do processamento do algoritmo



Fonte: O autor

As *tasks* criadas pelo FreeRTOS funcionam de forma independente do processamento sequencial, ou seja, são executadas sem a necessidade de um gatilho ou chamada no código principal, apenas executam sua função no período que foi determinada para ela e de acordo com sua prioridade da fila de tasks.

3.2.2. Inicialização do MCPWM

A inicialização do periférico MCPWM é definida após a configuração do tipo de modulação por parte de usuário. A habilitação das funções necessárias para geração de PWM trifásico requer algumas configurações específicas, baseado no conteúdo descrito na seção 2.7. Uma alternativa é alterar os bits dos registradores do MCPWM, outra opção é utilizar as funções disponibilizadas pela fabricante Espressif.

A Figura 29 exibe o código de configuração do *timer* e MCPWM. O *timer* é configurado para contagem crescente e decrescente, com a frequência de resolução máxima disponível para o MCPWM: 80 MHz. O parâmetro *period_ticks* recebe o número até qual o *timer* irá contar, que pode ser obtido a partir da frequência de resolução dividido pela frequência de modulação, escolhida como 20 kHz (50 μ s de período). Gerando um *timer* que conta até 4000 (2000 crescente e 2000 decrescente).

Figura 29 - Configuração do MCPWM timer e MCPWM operator

```

577     ESP_LOGI(TAG, "criando MCPWM timer");
578     mcpwm_timer_handle_t timer = NULL;
579     mcpwm_timer_config_t timer_config = {
580         .group_id = 0,
581         .clk_src = MCPWM_TIMER_CLK_SRC_DEFAULT,
582         .resolution_hz = freq_resolution_mcpwm,
583         .count_mode = MCPWM_TIMER_COUNT_MODE_UP_DOWN,
584         .period_ticks = dados_pwm.amplitude_senoide,
585     };
586     ESP_ERROR_CHECK(mcpwm_new_timer(&timer_config, &timer));
587
588     ESP_LOGI(TAG, "criando MCPWM operator");
589     mcpwm_oper_handle_t operators[3];
590     mcpwm_operator_config_t operator_config = {
591         .group_id = 0,
592     };
593     for (int i = 0; i < 3; i++){
594         ESP_ERROR_CHECK(mcpwm_new_operator(&operator_config, &operators[i]));
595     }
596
597     ESP_LOGI(TAG, "conectando operadores ao mesmo timer");
598     for (int i = 0; i < 3; i++){
599         ESP_ERROR_CHECK(mcpwm_operator_connect_timer(operators[i], timer));
600     }

```

Fonte: O autor

O restante do código, após a linha 588 descreve a criação do MCPWM *operator* e as suas configurações para que todos os operadores estejam relacionados ao mesmo *timer*, que foi criado anteriormente, isto é, executem ações em todos os operadores sempre que houver evento no único *timer* criado.

A Figura 30 apresenta o código da parametrização dos comparadores (*comparators*) e dos geradores (*generators*).

Figura 30 - Configuração do MCPWM comparator e MCPWM generator

```

604     ESP_LOGI(TAG, "criando comparadores");
605     mcpwm_cmpr_handle_t comparators[3];
606     mcpwm_comparator_config_t compare_config = {
607         .flags.update_cmp_on_tez = true,
608         .flags.update_cmp_on_tep = true,
609     };
610     for (int i = 0; i < 3; i++){
611         ESP_ERROR_CHECK(mcpwm_new_comparator(operators[i], &compare_config, &comparators[i]));
612         ESP_ERROR_CHECK(mcpwm_comparator_set_compare_value(comparators[i], 0)); // dados_pwm.amplitude_senoide / 2
613     }
614
615     ESP_LOGI(TAG, "criando MCPWM generators");
616     mcpwm_gen_handle_t generators[3][2] = {};
617     mcpwm_generator_config_t gen_config = {};
618     const int gen_gpios[3][2] = {
619         {fase_A_H, fase_A_L},
620         {fase_B_H, fase_B_L},
621         {fase_C_H, fase_C_L}
622     };
623
624     for (int i = 0; i < 3; i++){
625         for (int j = 0; j < 2; j++){
626             gen_config.gen_gpio_num = gen_gpios[i][j];
627             ESP_ERROR_CHECK(mcpwm_new_generator(operators[i], &gen_config, &generators[i][j]));
628         }
629     }

```

Fonte: O autor

A parametrização dos comparadores configura em quais eventos de contagem do timer o valor a ser comparado será atualizado por uma nova amostra a partir da *struct* *mcpwm_comparator_config_t*.

O *loop for* da linha 610 cria os comparadores e insere um valor inicial para ser comparado, que no código foi escolhido como zero para não gerar sinal PWM. A partir da linha 615 inicia-se a configuração dos geradores, que vão gerar as formas de onda PWM para os devidos pinos GPIO's do ESP32. A configuração é disposta de maneira que seja criado três pares de geradores, um para cada braço do inversor com seus dois transistores complementares cada.

A Figura 31 mostra a codificação para as ações no MCPWM *generator* e do MCPWM *dead time*.

Figura 31 - Configuração das ações do MCPWM *generator* e do *dead time*

```

629 ESP_LOGI(TAG, "setando acoes dos geradores");
630 for (int i = 0; i < 3; i++){
631     ESP_ERROR_CHECK(
632         mcpwm_generator_set_actions_on_compare_event(generators[i][0],
633             MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, comparators[i], MCPWM_GEN_ACTION_HIGH),
634             MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_DOWN, comparators[i], MCPWM_GEN_ACTION_LOW),
635             MCPWM_GEN_COMPARE_EVENT_ACTION_END()
636         )
637     );
638 }
639
640 ESP_LOGI(TAG, "configurando dead time");
641 mcpwm_dead_time_config_t config_tempo_morto = {
642     .posedge_delay_ticks = 80,
643     .negedge_delay_ticks = 0,
644     .flags.invert_output = true
645 };
646 for (int i = 0; i < 3; i++){
647     ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(generators[i][0], generators[i][0], &config_tempo_morto));
648 }
649
650 // saidas do PWM A e B serão complementares entre si
651 config_tempo_morto = (mcpwm_dead_time_config_t){
652     .posedge_delay_ticks = 0,
653     .negedge_delay_ticks = 80,
654     .flags.invert_output = false,
655 };
656 for (int i = 0; i < 3; i++){
657     ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(generators[i][0], generators[i][1], &config_tempo_morto));
658 }

```

Fonte: O autor

As ações do MCPWM *generator* são feitas com base no PWM digital descrito na Figura 22(b) e Figura 24, ou seja, quando o *timer* estiver incrementando e seu valor for igual ao valor a ser comparado, que será definido pela modulação SPWM ou SVM posteriormente, o sinal PWM irá para nível baixo e quando o *timer* estiver decrementando e seu valor for igual ao valor a ser comparado, o sinal PWM irá para nível alto. Estes são os eventos *UTEA* e *DTEA* descritos na Figura 24. Essa configuração é feita apenas para o PWMA de cada *generator* e pode ser feita também para o PWMB com a lógica invertida, para que sejam complementares, porém não será necessário, pois o módulo *dead time* se encarregará disso.

A partir da linha 640 o MCPWM *dead time* é configurado. O PWMA deve receber o delay de borda de subida (*rising edge delay*) e o PWMB recebe o delay de borda de descida (*falling edge delay*). Esse valor deve ser inserido como o número que o contador irá contar para

atingir o tempo morto necessário. Para uma frequência de resolução de 80 MHz e um tempo morto de 1 μ s, será necessário contar até 80. Por fim, a codificação da Figura 31 programa o tempo morto para ativo complementar baixo. Para mudar para ativo complementar alto basta inverter a flag de inversão do sinal de saída.

A Figura 32 exibe o código da programação da função de interrupção do timer do ESP32 para gerar o sinal de modulação, SPWM ou SVM.

Figura 32 - Configuração da função de interrupção

```

660 ESP_LOGI(TAG, "habilitando interrupção por timer para ajustar o duty cycle");
661 esp_timer_handle_t periodic_timer = NULL;
662 const esp_timer_create_args_t args_periodic_timers = {
663     .callback = mcpwm_isr_timer,
664     .arg = comparators,
665 };
666 ESP_ERROR_CHECK(esp_timer_create(&args_periodic_timers, &periodic_timer));
667 ESP_ERROR_CHECK(esp_timer_start_periodic(periodic_timer, 50));
668
669 ESP_LOGI(TAG, "iniciando o MCPWM_TIMER");
670 ESP_ERROR_CHECK(mcpwm_timer_enable(timer));
671 ESP_ERROR_CHECK(mcpwm_timer_start_stop(timer, MCPWM_TIMER_START_NO_STOP));

```

Fonte: O autor

Na estrutura *struct esp_timer_create_args_t* deve ser passado o nome da função de interrupção que será chamada a cada interrupção e os argumentos ou parâmetros que a função recebe, que nesse caso são os comparadores do MCPWM. Na função *esp_timer_start_periodic* é inserido o tempo em microssegundos de cada chamada de interrupção. Foi definido o tempo mínimo de interrupção que o ESP32 aceita, de 50 μ s para que a cada período de modulação do MCPWM fosse gerado uma nova amostra da onda senoidal, uma vez que 50 μ s de período corresponde a 20 kHz de frequência.

A Figura 33 apresenta a codificação feita para atualizar o sinal senoidal que serve de referência (sinal modulante) para gerar o PWM. A variável *status_modulacao* define o tipo de modulação que será utilizada (SPWM ou SVM). Caso esteja habilitado para modulação SVM, o microcontrolador irá calcular as senoides correspondentes aos eixos alfa e beta e entregar para a função *svm_run* que pode ser encontrada no ANEXO B. Essa função irá executar o algoritmo de identificação dos setores e devolver o período de modulação de cada chave.

Figura 33 - Função de interrupção para atualização do sinal de modulação

```

92 void IRAM_ATTR mcpwm_isr_timer(void *arg)
93 {
94     static int cont_amostras = 0;
95     if (mystatus.status_modulacao){
96         space_vector.Ualpha = sin(dados_pwm.ohmega * cont_amostras);
97         space_vector.Ubeta = cos(dados_pwm.ohmega * cont_amostras);
98
99         svm_run(&space_vector);
100
101         space_vector.Ta = dados_pwm.amplitude_senoide * (1 + dados_pwm.indice_modulacao * space_vector.Ta) * 0.25;
102         space_vector.Tb = dados_pwm.amplitude_senoide * (1 + dados_pwm.indice_modulacao * space_vector.Tb) * 0.25;
103         space_vector.Tc = dados_pwm.amplitude_senoide * (1 + dados_pwm.indice_modulacao * space_vector.Tc) * 0.25;
104
105         WRITE_PERI_REG(0x3FF5E040, space_vector.Ta);
106         WRITE_PERI_REG(0x3FF5E078, space_vector.Tb);
107         WRITE_PERI_REG(0x3FF5E0B0, space_vector.Tc);
108     }
109     else{
110         spwm_gen(&dados_pwm, &spwm, cont_amostras);
111
112         WRITE_PERI_REG(0x3FF5E040, spwm.valor_fase_A);
113         WRITE_PERI_REG(0x3FF5E078, spwm.valor_fase_B);
114         WRITE_PERI_REG(0x3FF5E0B0, spwm.valor_fase_C);
115     }
116     cont_amostras += 1;
117     if (cont_amostras >= dados_pwm.num_amostras){
118         cont_amostras = 0;
119     }
120 }
121

```

Fonte: O autor

Após obter o período de modulação de cada chave (sinal entre -1 e 1), será necessário calcular o sinal modulante normalizado descrito na equação (16). Ressalta-se que a variável *amplitude_senoide* armazena a informação do valor de contagem do timer (parte crescente e decrescente), portanto deve-se dividir o valor final por 2, por esse motivo tem-se um fator de 0,25. A função *WRITE_PERI_REG* serve para escrever uma informação em um registrador diretamente pelo endereço de memória do registrador. Nesse caso, ela foi utilizada para escrever nos endereços dos registradores sombra *PWM_GENx_TSTMP_A_REG* onde é armazenado o valor do PWM antes de ir para os comparadores. De maneira semelhante, é feito o mesmo para o caso da modulação SPWM, conforme descrito no fluxograma da Figura 28.

Por fim, é incrementado o contador de amostras. O número de amostras varia de acordo com a frequência da senoide a ser gerada e depende da frequência de modulação, por exemplo para uma senoide de 100 Hz e a frequência de modulação de 20 kHz, obtém-se 200 amostras por ciclo completo da senoide.

Assim, sempre que a contagem de amostras atinge o número de amostras, ela é zerada novamente pois inicia-se um novo ciclo senoidal. O número de amostras pode ser modificado após a conversão analógica para digital, ao obter a frequência da senoide que o usuário deseja modular. Ela é obtida como sendo a razão entre a frequência de modulação e a frequência da onda senoidal.

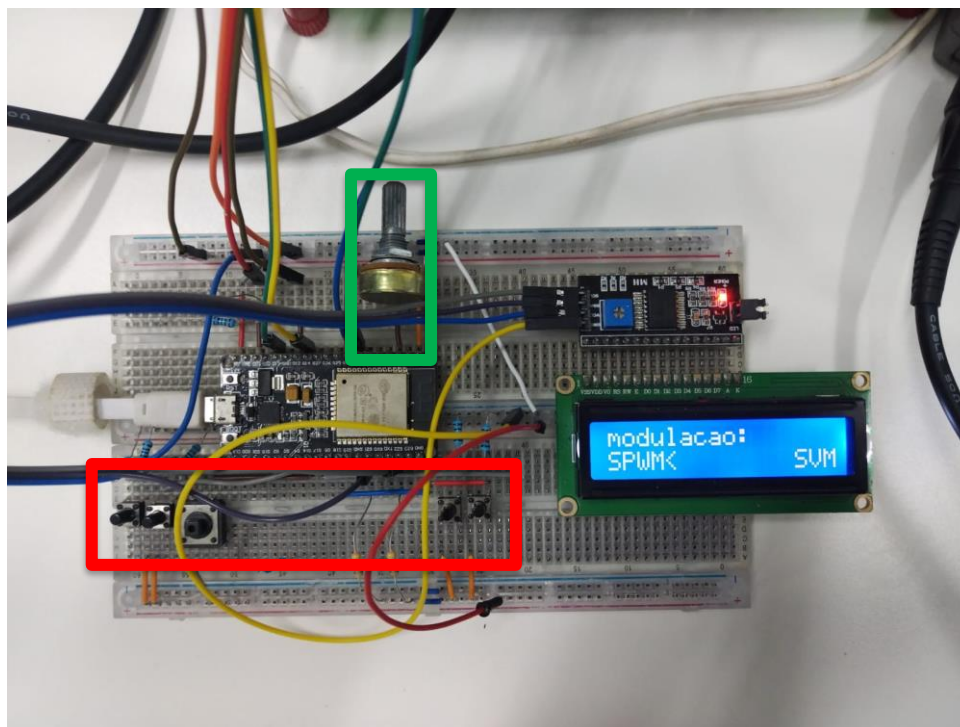
4. RESULTADOS

4.1. Implementação do sistema digital de acionamento do inversor trifásico

4.1.1. Protótipo do circuito de acionamento

A Figura 34 exhibe como o circuito de controle do sinal PWM foi montado.

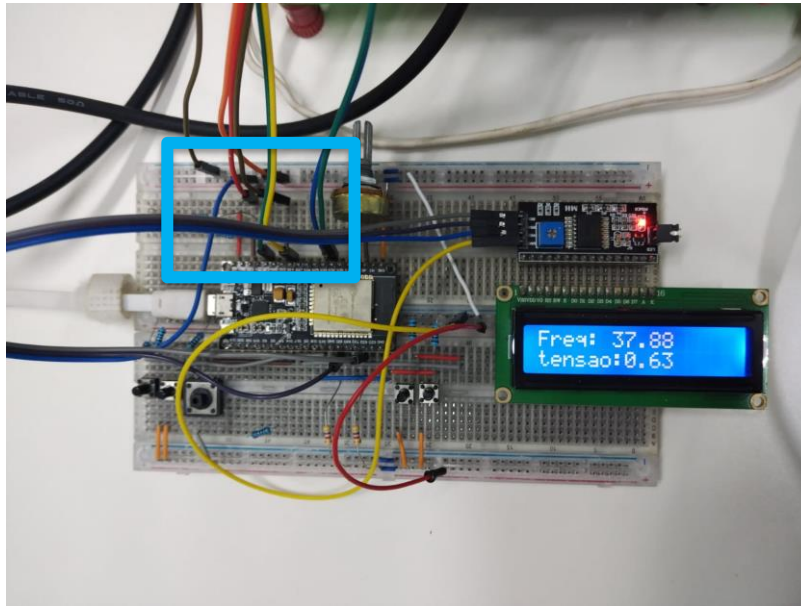
Figura 34 - Circuito de acionamento do inversor e configuração do tipo de modulação



Fonte: O autor

O circuito conta com o microcontrolador ESP32, botões (destacados no retângulo vermelho) para realizar as configurações no menu, um display LCD para exibir o menu e um potenciômetro (destacado no retângulo verde) para modificação analógica da frequência. A Figura 35 mostra o menu de ajuste de frequência feito pelo microcontrolador.

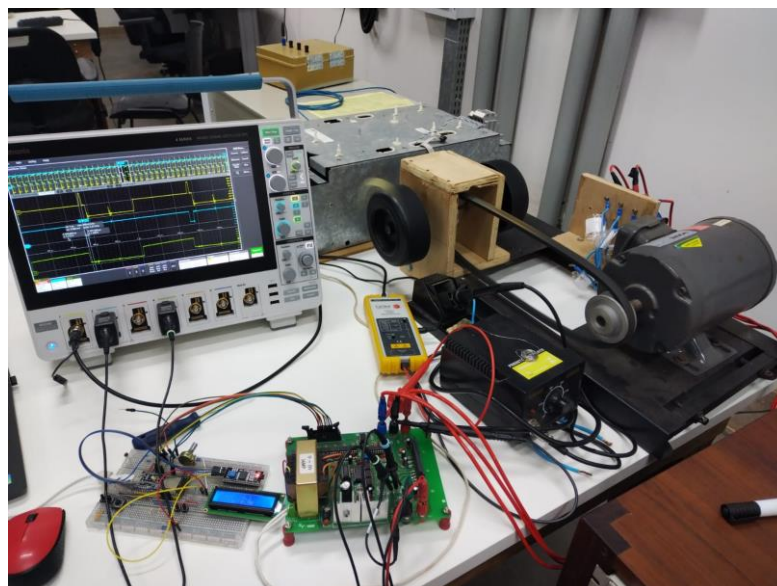
Figura 35 - Ajuste da frequência



Fonte: O autor

O retângulo azul claro destaca os seis pinos de PWM que vão para a placa inversora. A Figura 36 exibe o circuito de controle, a placa inversora, o motor de indução e o osciloscópio para medições.

Figura 36 - Circuito de controle e de potência

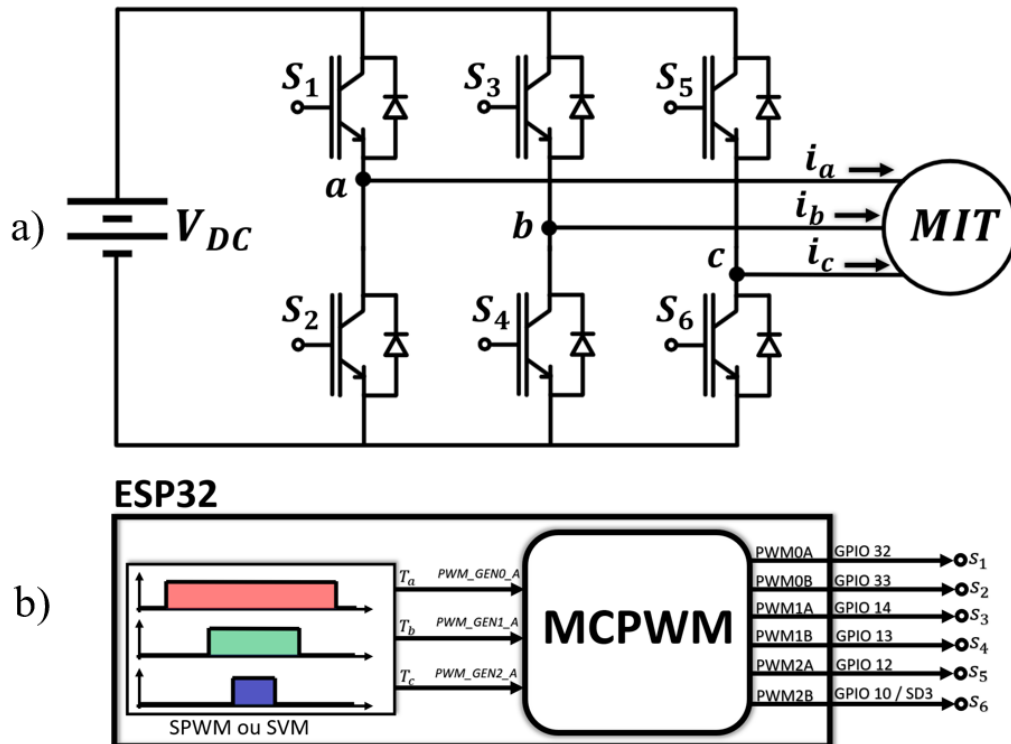


Fonte: O autor

A placa inversora recebe tensão de uma fonte CC para alimentação do motor de indução através dos transistores. O microcontrolador envia os sinais PWM para a placa inversora para

chaveamento dos transistores. O motor de indução trifásico também está conectado a placa inversora. A Figura 37 exibe o diagrama esquemático do circuito do projeto, mostrando em quais pinos do ESP32 os sinais PWM são gerados, que são transmitidos aos transistores do inversor por meio de fotoacopladores (sendo estes omitidos no esquemático).

Figura 37 - Esquemático do inversor: a) sistema de potência, b) sistema microcontrolado



Fonte: O autor

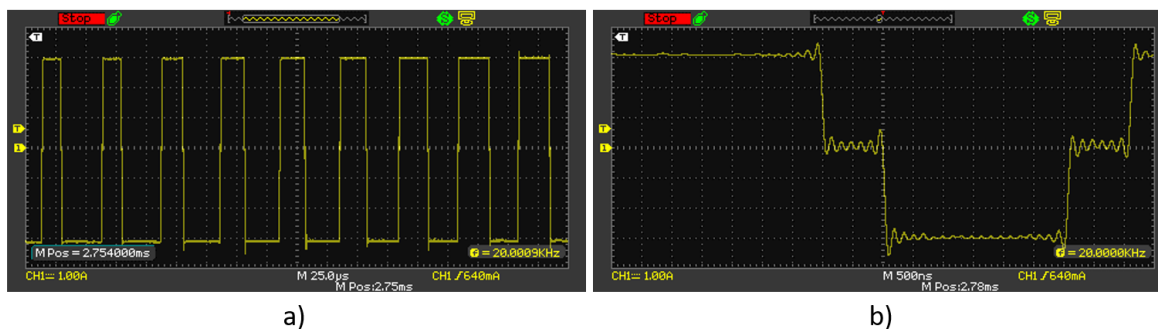
O módulo MCPWM do microcontrolador ESP32 recebe nos registradores `PWM_GEN0_TSTMP_A_REG` (endereço `0X3FF5E040`), `PWM_GEN1_TSTMP_A_REG` (endereço `0X3FF5E078`) e `PWM_GEN2_TSTMP_A_REG` (endereço `0X3FF5E0B0`) a informação do período de modulação respectivamente para cada fase. O módulo MCPWM processa essas informações de acordo com as configurações realizadas nos módulos descritos na seção 2.7 e entrega em sua saída os 6 sinais PWM, que estarão conectados em pinos GPIO do próprio microcontrolador. No código do algoritmo (ANEXO A) esses registradores são acessados diretamente pela função `WRITE_PERI_REG`.

4.2. Resultados experimentais

A Figura 38(a) exibe a forma de onda de tensão entre os sinais de PWM de um braço superior e do mesmo braço inferior e a Figura 38(b) exibe o tempo morto entre o desligamento

de um PWM e o acionamento de outro. Esses dados foram medidos a partir dos pinos PWM0A (GPIO 32) e PMW0B (GPIO33) visíveis no esquemático do inversor na Figura 37(b).

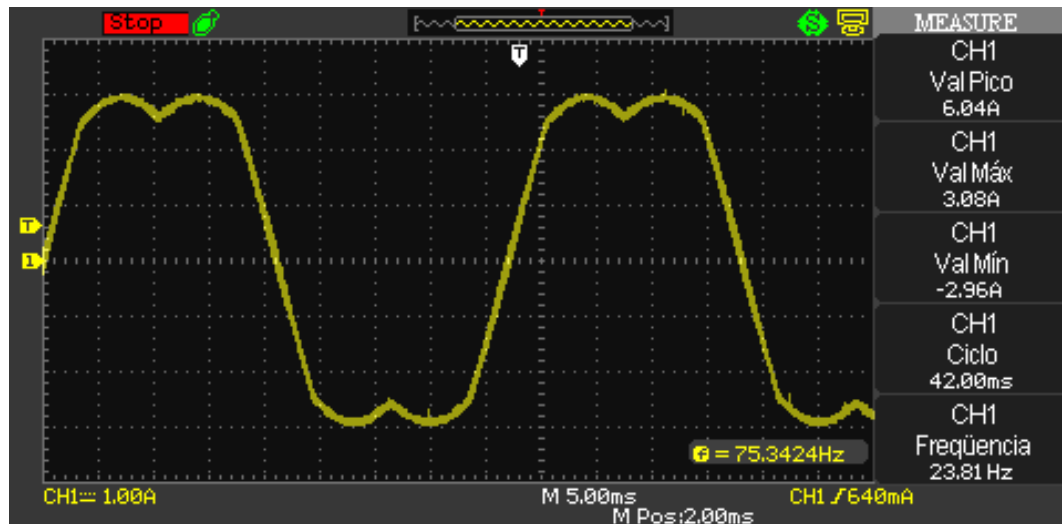
Figura 38 – Forma de onda da saída dos PWM's de um braço: a) Tensão entre PWM superior e inferior de um braço, b) tempo morto



Fonte: O autor

É possível verificar pelas medições que a frequência de chaveamento é de 20 KHz, conforme a que foi programada no microcontrolador. O microcontrolador foi programado para gerar um tempo morto de 1 μ s. O tempo em nível baixo da Figura 38(b) corresponde a esse valor parametrizado pelo MCPWM. A forma de onda da Figura 38(a) gera o sinal característico da onda modulante vista na Figura 15. O sinal mostrado na Figura 15 se refere à uma variável interna do microcontrolador e, para viabilizar a visualização desta variável em um osciloscópio, foi utilizado um pino de PWM adicional configurado para a frequência de 20 KHz e um filtro passa-baixa de primeira ordem sintonizado em 1 kHz, desta forma, o referido sistema se comporta como um conversor analógico para digital, sendo possível visualizar essa forma de onda, conforme Figura 39.

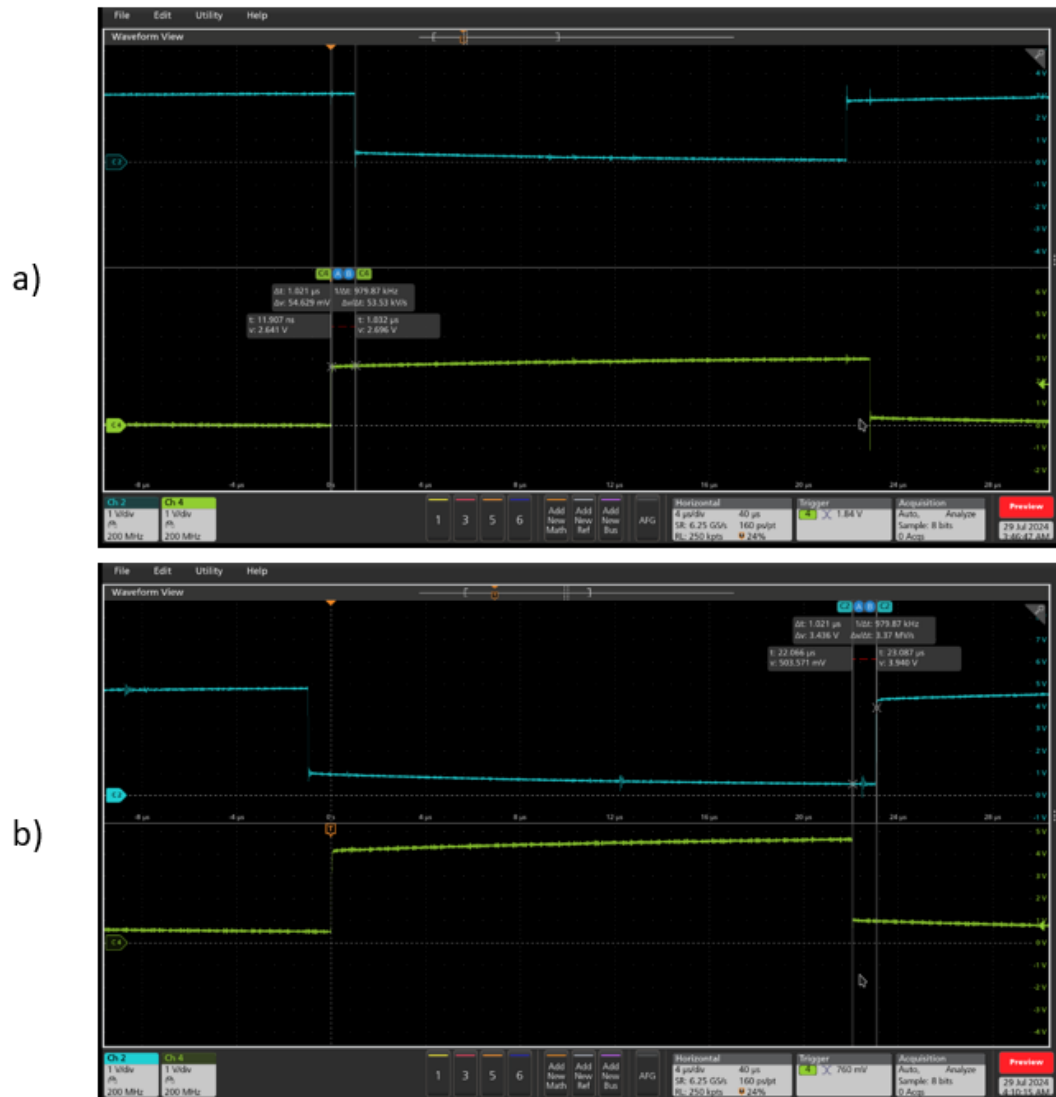
Figura 39 - Forma de onda característica do SVM



Fonte: O autor

A placa inversora gera um sinal invertido em relação ao sinal PWM gerado pelo microcontrolador, devido a portas lógicas *NAND* presente nos fotoacopladores, portanto na técnica de tempo morto ativo complementar alto, os transistores do mesmo braço estarão ligados simultaneamente (do ponto de vista do sinal gerado pelo microcontrolador). Para contornar esse problema, basta fazer com que o microcontrolador gere o tempo morto ativo complementar baixo. A Figura 40 mostra esse efeito.

Figura 40 - Sinal PWM: a) tempo morto ativo complementar baixo gerado pelo ESP32, b) tempo morto ativo complementar alto da placa inversora

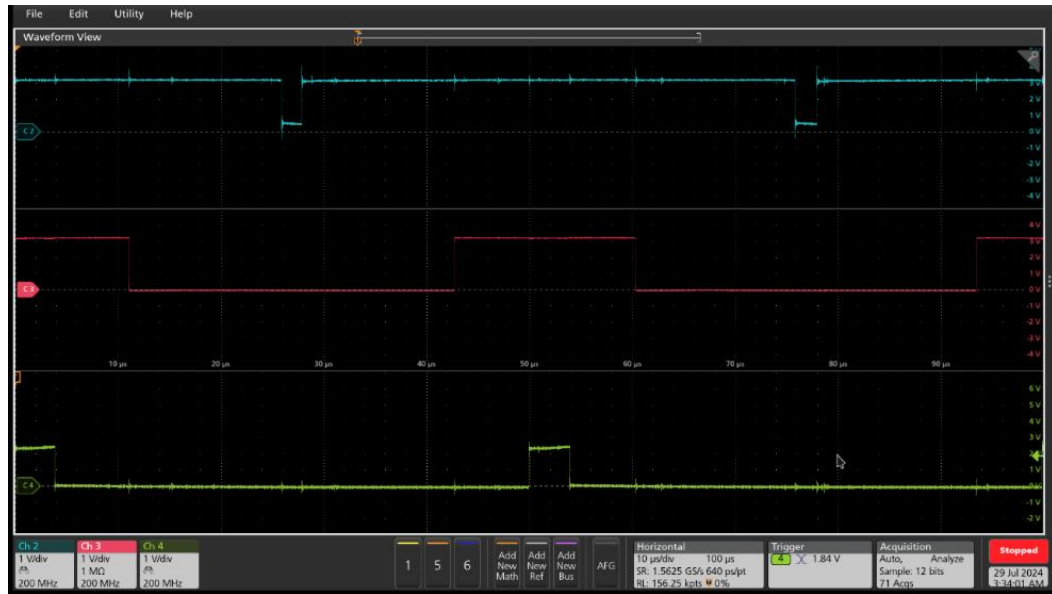


Fonte: O autor

As formas de onda da Figura 40(a) foram tomadas nos pinos de saída GPIO32 e GPIO33 do ESP32 na Figura 37(b), enquanto as da Figura 40(b) no circuito da placa inversora, na saída dos ponto S_1 e S_2 na Figura 37(a).

A Figura 41 exibe o sinal PWM do transistor superior das três fases do inversor trifásico. Os sinais medidos nos pinos PWM do microcontrolador se assemelham aos da Figura 12, constatando a geração do sinal PWM simétrico pelo SVM.

Figura 41 – Forma de onda do PWM trifásico simétrico gerado pelo SVM

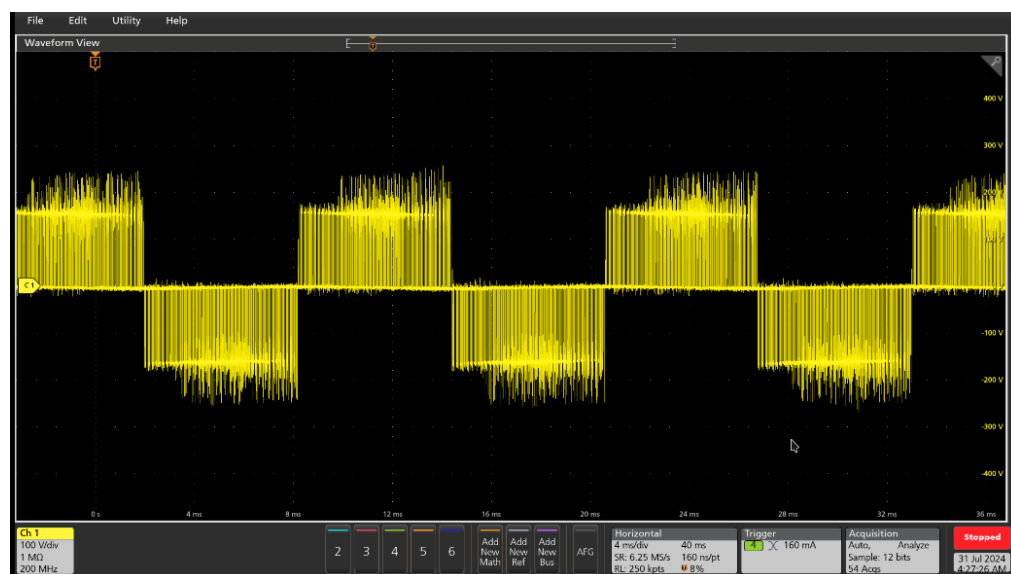


Fonte: O autor

Os dados dessa forma de onda foram obtidos medindo os pinos do PWM0A, PWM1A e PWM2A descritos na Figura 37(b).

A Figura 42 mostra a forma de onda da tensão de linha V_{ab} . Esta tensão é tomada no ponto em que é conectado o motor de indução, na parte central do braço do inversor, ou seja, nos pontos a e b do esquemático do inversor na Figura 37(a).

Figura 42 – Forma de onda da tensão V_{ab} a 4 ms/div com SVM.



Fonte: O autor

A Figura 43 exibe a forma de onda da corrente elétrica em cada fase do motor de indução trifásico e a tensão V_{ab} aplicando o SPWM para uma frequência de 45 Hz no motor. Os dados foram coletados com medidores de corrente e um medidor de tensão. Os medidores foram conectados na fiação na entrada do motor, conforme é possível identificar na Figura 37(a).

Figura 43 – Forma de onda da corrente elétrica no motor com o SPWM

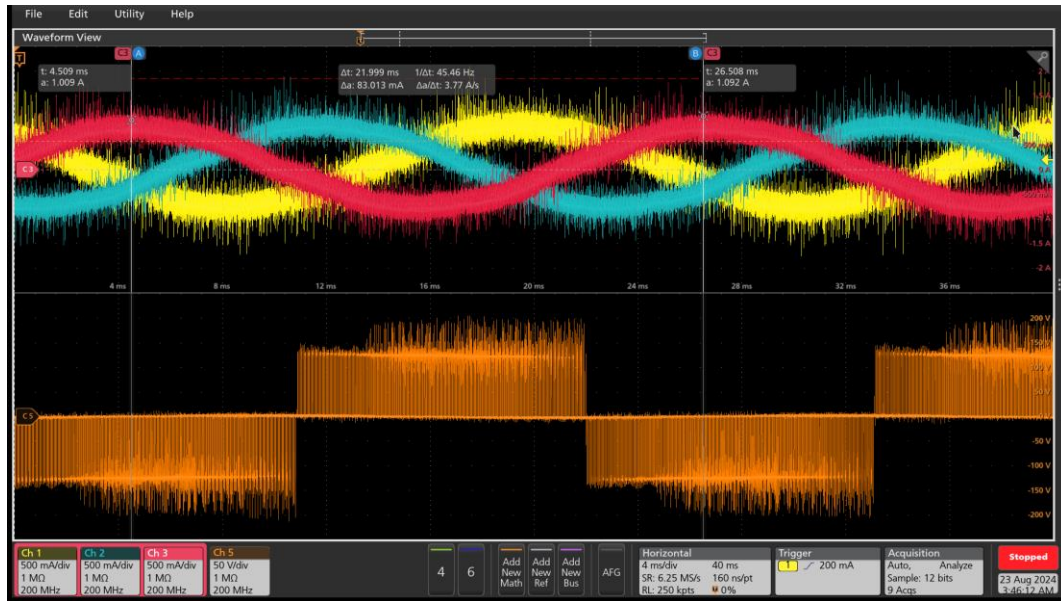


Fonte: O autor

Em laranja tem-se a tensão de fase V_{ab} com aproximadamente 150 V de amplitude. É possível perceber que as ondas são praticamente senoidais e equilibradas, com amplitude semelhante e defasadas de 120°. O sinal apresenta ruído, devido ao chaveamento dos transistores. O motor, com característica indutiva, se comporta como um filtro passa baixa, atenuando os sinais de alta frequência, fazendo com que a forma de onda da corrente se assemelhe à modulante, com algum *ripple* distorcendo levemente o sinal.

A Figura 44 exibe a forma de onda da corrente em cada fase do motor e a tensão V_{ab} aplicando o SVM para uma frequência de 45 Hz. É possível notar que para a mesma frequência e nível de tensão, obtêm-se um valor de amplitude de corrente elétrica ligeiramente maior, cerca de 15%, conforme o que foi estudado na fundamentação teórica.

Figura 44 – Forma de onda da corrente elétrica no motor com SVM



Fonte: O autor

A Figura 45 exibe a corrente no motor com uma frequência de 70 Hz e a tensão da fonte V_{DC} aproximadamente de 250 V.

Figura 45 - Corrente elétrica no motor com SVM a 250 V_{DC}



Fonte: O autor

Mesmo para tensões próximas da tensão nominal, o SVM apresentou excelentes resultados na geração da onda senoidal para alimentar um motor de 0,5 cv e fazê-lo girar, mesmo que ainda a vazio.

5. CONCLUSÕES

O objetivo desse trabalho foi a geração de tensão senoidal para inversor de frequência trifásico aplicado a motores de indução a partir de modulação PWM com SPWM e SVM utilizando o microcontrolador ESP32 com seu periférico dedicado MCPWM.

Com as formas de onda observadas, em análise e comparação com a fundamentação teórica constata-se que os objetivos do projeto proposto foram alcançados com sucesso. Conforme o ANEXO A, foi possível programar um código no microcontrolador ESP32 que realizasse o chaveamento do inversor usando a modulação PWM necessária para acionar um motor de indução utilizando seu periférico dedicado MCPWM embarcado no microcontrolador ESP32. Além disso, foram implementados os algoritmos de geração de tensão senoidal nesse dispositivo moderno e barato: tanto o SPWM como o SVM. Por fim, foram realizadas capturas das formas de onda vistas na fundamentação teórica, como a onda característica da modulação vetorial espacial, tempo morto, sinais PWM e corrente no motor de indução, bem como a observação do motor girando em laboratório.

Em nível de graduação, este ainda é um estudo inovador a respeito da aplicação do ESP32 para fins de eletrônica de potência. Podendo trazer boas contribuições para trabalhos futuros com o periférico MCPWM devido ao conteúdo teórico a respeito da utilização deste microcontrolador e sobre a modulação SVM.

5.1. Sugestões para trabalhos futuros

- Medição em tempo real da rotação do motor para controle em malha fechada;
- Usar sensores de corrente para aplicação de controle orientado a campo (FOC);
- Melhorias no menu inserindo frenagem, mudança no sentido de rotação, modificação da frequência a partir de botões, além do potenciômetro;
- Estudar outras aplicações em eletrônica de potência, como uso do MCPWM para controle de motores BLDC e conversores DC-DC isolados que utilizam ponte H.

REFERÊNCIAS

BRITO, Eduardo Carvalho da Rocha. **Controle vetorial para acionamento de máquinas de indução**. 2014. Trabalho de Conclusão de Curso.

SIMÕES FILHO, Elson Amorim. Estudo do PWM espaço vetorial para o inversor de dois-níveis para carga trifásica. 2019.

BUSO, Simone; MATTAVELLI, Paolo. **Digital control in power electronics**. Morgan & Claypool Publishers, 2015.

RAMOS, Guilherme Michel Alves de. **Desenvolvimento de um inversor trifásico de tensão controlado por um processador digital de sinal**. 2018. Trabalho de Conclusão de Curso. Universidade Tecnológica Federal do Paraná.

FERRONI, Felipe Martins. Inversor de tensão senoidal microcontrolado por Arduino. 2018.

GURGEL, Nathan et al. Transformações em Sistemas Elétricos de Potência: Análise das Transformadas de Clarke e Park. **Revista Eletrônica de Engenharia Elétrica e Engenharia Mecânica**, v. 3, n. 1, p. 01-12, 2021.

FORTESCUE, Charles L. Method of symmetrical co-ordinates applied to the solution of polyphase networks. **Transactions of the American Institute of Electrical Engineers**, v. 37, n. 2, p. 1027-1140, 1918.

KART, Diagram Source Brain. Pulse width modulation. 2001.

FRANCHI, Claiton Moro. **Inversores de Frequência: Teoria e Aplicações**. 2. ed. São Paulo: Editora Érica, 2013.

Manual de Referência Técnica do ESP32. [s.l: s.n.]. Disponível em:

<https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf>.

Documentos Técnicos | Espressif Systems. Disponível em:

<<https://www.espressif.com/en/support/documents/technical-documents>>.

MCPWM - ESP32 - — ESP-IDF Documentação do Guia de Programação v4.3. Disponível em:

<<https://docs.espressif.com/projects/esp-idf/en/v4.3/esp32/api-reference/peripherals/mcpwm.html>>. Acesso em: 10 jul. 2024.

Motor Control Pulse Width Modulator (MCPWM) - ESP32 - — ESP-IDF Programming Guide latest documentation. Disponível em: <<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/mcpwm.html#>>. Acesso em: 10 jul. 2024.

API Conventions - ESP32 - — ESP-IDF Programming Guide v5.2.2 documentation. Disponível em: <<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/api-conventions.html>>. Acesso em: 15 jul. 2024.

DE GASPERI, Daniel. **APLICAÇÃO DA MODULAÇÃO VETORIAL ESPACIAL PARA O CONTROLE DE INVERSORES TRIFÁSICOS DE DOIS E TRÊS NÍVEIS.** 2019. Tese de Doutorado. Universidade Federal do Rio de Janeiro.

DE ARAUJO SILVA, Leonardo. Avaliação de uma nova proposta de controle v/f em malha aberta. **Master's thesis, UNICAMP**, 2000.

GARCIA, EDUARDO VISONI. Estudo do método de controle de velocidade do tipo V/F em máquinas de indução utilizando o Simulink.

BERTOLUCCI, Gabriel Cardoso et al. Modelagem e simulação de controle escalar de velocidade de um motor de indução trifásico. 2022.

Space Vector PWM Intro. Disponível em: <<https://www.switchcraft.org/learning/2017/3/15/space-vector-pwm-intro>>.

SAENZ, Wohler Gonzales; QUISPE, Richard Corasma. Modulador por anchura de pulso sinusoidal SPWM para un inversor monofásico de 60Hz de bajo costo. **Polo del Conocimiento: Revista científico-profesional**, v. 6, n. 5, p. 179-191, 2021.

TREVISO, C. H. G., **Apostila de eletrônica de Potência**, 2006.

NEACSU, Dorin O. Modulação de vetor espacial - Uma introdução. Em: **IECON**. 2001. p. 1583-1592.

FreeRTOS Documentation - FreeRTOS™. Disponível em: <https://www.freertos.org/Documentation/02-Kernel/07-Books-and-manual/01-RTOS_book>.

ANEXOS

ANEXO A – Código principal utilizado para programação da modulação do inversor

```
#include <stdio.h>
#include "esp_log.h"
#include "esp_system.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "soc/mcpwm_reg.h"
#include "soc/mcpwm_struct.h"
#include "driver/mcpwm_prelude.h"
#include "fastmath.h"
#include "esp_timer.h"
#include "HD44780.h"
#include "gerador_senoide.h"
#include "spi_flash_mmap.h"
#include "esp_adc/adc_oneshot.h"

static const char *TAG = "projeto PWM senoidal trifasico";

// braços do inversor
const int fase_A_H = 32; // G32
const int fase_A_L = 33; // G33
const int fase_B_H = 14; // G14
const int fase_B_L = 13; // G13
const int fase_C_H = 12; // G12
const int fase_C_L = 10; // SD3

#define botao_subir GPIO_NUM_15
#define botao_descer GPIO_NUM_16
#define botao_esquerda GPIO_NUM_17
#define botao_direita GPIO_NUM_18
```



```

#define botao_confirma GPIO_NUM_19
#define botao_cancela GPIO_NUM_23
#define botao_incrmt GPIO_NUM_2
#define botao_decrmt GPIO_NUM_4

const uint8_t botoes[8] = {botao_subir, botao_descer, botao_esquerda, botao_direita,
botao_confirma, botao_cancela, botao_incrmt, botao_decrmt};
uint8_t botao_lido;

#define freq_resolution_mcpwm 80E6 // frequência de 80Mhz do clock que vai para o mcpwm
#define pi 3.1415926536

#define LCD_ADDR 0x27
#define LCD_ROWS 2
#define LCD_COLS 16
#define PIN_SCL 22
#define PIN_SDA 21

#define EXAMPLE_ADC1_CHAN0 ADC_CHANNEL_0

adc_oneshot_unit_handle_t adc1_handle;
adc_oneshot_unit_init_cfg_t init_config1 = {
    .unit_id = ADC_UNIT_1,
};

typedef struct{
    bool status_modulacao; //true ativa a modulação vetorial espacial
    bool status_contrl_pot;
    bool status_contrl_discreto;
    bool status_contrl_partd_suave;
    bool status_timer_mcpwm;
    bool status_ativar_mcpwm;
} status_t;

```

```

typedef struct{
    int dado_cad;
    float freq_cad;

    char string_num[10];
} dados_cad_t;

svm_t space_vector;
pwm_senoidal_t spwm;
dados_modulacao_t dados_pwm;
dados_cad_t dados_cad;
status_t mystatus;

//struct de configuração do timer do MCPWM
mcpwm_timer_handle_t timer = NULL;

/*
=====

                        FUNÇÃO DE INTERRUPÇÃO

=====
*/

void IRAM_ATTR mcpwm_isr_timer(void *arg)
{
    static int cont_amostras = 0;

    if (mystatus.status_modulacao)
    {
        space_vector.Ualpha = sin(dados_pwm.ohmega * cont_amostras);
        space_vector.Ubeta = cos(dados_pwm.ohmega * cont_amostras);

        svm_run(&space_vector);
    }
}

```

```

    space_vector.Ta = dados_pwm.amplitude_senoide * (1 + dados_pwm.indice_modulacao
* space_vector.Ta) * 0.25;
    space_vector.Tb = dados_pwm.amplitude_senoide * (1 + dados_pwm.indice_modulacao
* space_vector.Tb) * 0.25;
    space_vector.Tc = dados_pwm.amplitude_senoide * (1 + dados_pwm.indice_modulacao
* space_vector.Tc) * 0.25;

```

```

    WRITE_PERI_REG(0x3FF5E040, space_vector.Ta);
    WRITE_PERI_REG(0x3FF5E078, space_vector.Tb);
    WRITE_PERI_REG(0x3FF5E0B0, space_vector.Tc);
}

```

```

else

```

```

{
    spwm_gen(&dados_pwm, &spwm, cont_amostras);

```

```

    WRITE_PERI_REG(0x3FF5E040, spwm.valor_fase_A);
    WRITE_PERI_REG(0x3FF5E078, spwm.valor_fase_B);
    WRITE_PERI_REG(0x3FF5E0B0, spwm.valor_fase_C);
}

```

```

cont_amostras += 1;
if (cont_amostras >= dados_pwm.num_amostras)
{
    cont_amostras = 0;
}

```

```

}

```

```

/*

```

```

=====

```

FUNÇÕES UTILIZADAS

```

=====

```

```

*/

```

```

void inicia_mcpwm();
void ler_botoes(void *pvParameter);
void menu(void *pvParameter);
void task1(void *pvParameter);
void conversao_AD(void *pvParameter);
void controle_de_freq(void *pvParameter);
void partida_suave(void *pvParameter);

/*
=====

                        CONFIGURAÇÃO DO PROGRAMA E LOOP PRINCIPAL

=====
*/

void app_main(void) // configurações iniciais
{
    // configurações do pwm e da geração da senóide
    //-----

    dados_pwm.frequencia_modulacao = 20000; // frequência de 20kHz para a onda triangular
    gerada pelos timers
    dados_pwm.periodo_modulacao = 50E-6;
    dados_pwm.freq_senoide = 60.0;
    dados_pwm.freq_inicial = 6.0;
    dados_pwm.indice_modulacao = 0.1;
    // valor da amplitude da onda senoidal (mesmo do valor máx do período do timer da
    triangular)
    dados_pwm.amplitude_senoide = freq_resolution_mcpwm /
    dados_pwm.frequencia_modulacao;
    dados_pwm.ohmega = 2 * pi * dados_pwm.freq_inicial * dados_pwm.periodo_modulacao;
    dados_pwm.num_amostras = dados_pwm.frequencia_modulacao / dados_pwm.freq_inicial;

    mystatus.status_modulacao = true;
    mystatus.status_contrl_pot = false;
    mystatus.status_contrl_discreto = false;

```

```

mystatus.status_contrl_partd_suave = false;
mystatus.status_timer_mcpwm = false;

//-----

lcd_init();
lcd_clear();
lcd_set_cursor(0, 0);
lcd_send_string("lcd i2c ok");
// vTaskDelay(1000 / portTICK_PERIOD_MS);

//-----

// configuração conversão analógica digital

ESP_ERROR_CHECK(adc_oneshot_new_unit(&init_config1, &adc1_handle));

adc_oneshot_chan_cfg_t config = {
    .bitwidth = ADC_BITWIDTH_DEFAULT,
    .atten = ADC_ATTEN_DB_12,
};
ESP_ERROR_CHECK(adc_oneshot_config_channel(adc1_handle,
EXAMPLE_ADC1_CHAN0, &config));

//-----

esp_rom_gpio_pad_select_gpio(GPIO_NUM_9);
gpio_set_direction(GPIO_NUM_9, GPIO_MODE_OUTPUT);
esp_rom_gpio_pad_select_gpio(GPIO_NUM_25);
gpio_set_direction(GPIO_NUM_25, GPIO_MODE_OUTPUT);
//-----

xTaskCreate(task1, "task1", 1024, NULL, 5, NULL);
xTaskCreatePinnedToCore(ler_botoes, "ler_botoes", 2048, NULL, 4, NULL, 1);
xTaskCreatePinnedToCore(menu, "menu", 2048, NULL, 4, NULL, 1);

```

```

xTaskCreatePinnedToCore(conversao_AD, "conversao_AD", 2048, NULL, 4, NULL, 1);
xTaskCreatePinnedToCore(controle_de_freq, "controle_de_freq", 2048, NULL, 4, NULL,
1);
xTaskCreatePinnedToCore(partida_suave, "partida_suave", 2048, NULL, 4, NULL, 1);

//-----

while (1){
    // MCPWM foi ativado no menu
    if(mystatus.status_timer_mcpwm){
        inicia_mcpwm();

        mystatus.status_timer_mcpwm = false;
    }
    vTaskDelay(500 / portTICK_PERIOD_MS);
}
} // end main

/*
=====
                        FUNÇÕES UTILIZADAS
=====
*/

void conversao_AD(void *pvParameters) {
    // Tamanho da janela de média móvel
    static const int tamanho_janela = 50;
    static uint16_t dados_cad_FMM[50] = {0.0};
    static float soma_valores = 0.0;
    static int posicao_atu = 0;
    static bool janela_cheia = false;

    while (1) {

```

```
ESP_ERROR_CHECK(adc_oneshot_read(adc1_handle, EXAMPLE_ADC1_CHAN0,
&dados_cad.dado_cad));
```

```
// Adicionar o novo valor à soma
```

```
soma_valores += dados_cad.dado_cad;
```

```
// Se a janela estiver cheia, subtrair o valor mais antigo
```

```
if (janela_cheia) {
```

```
    soma_valores -= dados_cad_FMM[posicao_atu];
```

```
}
```

```
// Armazenar o novo valor no buffer de média móvel
```

```
dados_cad_FMM[posicao_atu] = dados_cad.dado_cad;
```

```
// Calcular a média móvel
```

```
float media_movel = soma_valores / (janela_cheia ? tamanho_janela : (posicao_atu + 1));
```

```
// Atualizar a frequência com base na média móvel
```

```
dados_cad.freq_cad = (float)media_movel * 114 * 0.000244200244 + 6;
```

```
// Atualizar a posição na janela
```

```
posicao_atu = (posicao_atu + 1) % tamanho_janela;
```

```
// Verificar se a janela está cheia
```

```
if (posicao_atu == 0) {
```

```
    janela_cheia = true;
```

```
}
```

```
// Converter a frequência para string
```

```
sprintf(dados_cad.string_num, "%.2f", dados_cad.freq_cad);
```

```
// Delay para a próxima leitura
```

```
vTaskDelay(150 / portTICK_PERIOD_MS);
```

```
}
```

```
}
```

```
void controle_de_freq(void *pvParameters){
    char string_indice_mod[10];
    while (1)
    {
        while (mystatus.status_contrl_pot == true && mystatus.status_contrl_partd_suave ==
false)
        {
            dados_pwm.ohmega = 2 * pi * dados_cad.freq_cad * dados_pwm.periodo_modulacao;
            dados_pwm.num_amostras = (int)dados_pwm.frequencia_modulacao /
dados_cad.freq_cad;

            // se a frequência for menor que a frequência nominal da senoidal
            // ajustar o índice de modulação
            if (dados_cad.freq_cad <= dados_pwm.freq_senoide)
            {
                dados_pwm.indice_modulacao = dados_cad.freq_cad / dados_pwm.freq_senoide;

                lcd_set_cursor(1, 0);
                lcd_send_string("tensao:   ");
                sprintf(string_indice_mod, "%.2f", dados_pwm.indice_modulacao);
                lcd_set_cursor(1, 7);
                lcd_send_string(string_indice_mod);
            }
            else
            {
                dados_pwm.indice_modulacao = 0.9;
                lcd_set_cursor(1, 0);
                lcd_send_string("tensao:   ");
                sprintf(string_indice_mod, "%.2f", dados_pwm.indice_modulacao);
                lcd_set_cursor(1, 7);
                lcd_send_string(string_indice_mod);
            }
        }
    }
}
```



```

    //lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_send_string("Freq:      ");
    lcd_set_cursor(0, 6);
    lcd_send_string(dados_cad.string_num);
    vTaskDelay(200 / portTICK_PERIOD_MS);
}

while (mystatus.status_contrl_discreto == true && mystatus.status_contrl_partd_suave ==
false){
    lcd_set_cursor(0, 0);
    lcd_send_string("nada para fazer ");
    lcd_set_cursor(1, 0);
    lcd_send_string("nada para fazer ");

    vTaskDelay(200 / portTICK_PERIOD_MS);
}
vTaskDelay(600 / portTICK_PERIOD_MS);
}
}

void partida_suave(void *pvParameters){
    float freq_ref = 60.0;
    static float freq_inicial = 6.0;
    char string_indice_mod[10];
    char string_freq[10];

    while (1){
        while(mystatus.status_contrl_partd_suave == true && mystatus.status_timer_mcpwm ==
false){
            if(mystatus.status_contrl_pot){
                freq_ref = dados_cad.freq_cad;
            }

```

```

else{
    // valor da frequência selecionado através dos botões
    freq_ref = 60.0;
}

dados_pwm.ohmega = 2 * pi * freq_inicial * dados_pwm.periodo_modulacao;
dados_pwm.num_amostras = (int)dados_pwm.frequencia_modulacao / freq_inicial;

// se a frequência for menor que a frequência nominal da senoidal
// ajustar o índice de modulação
if (freq_inicial <= dados_pwm.freq_senoide)
{
    dados_pwm.indice_modulacao = freq_inicial / dados_pwm.freq_senoide;

    lcd_set_cursor(1, 0);
    lcd_send_string("tensao:   ");
    sprintf(string_indice_mod, "%.2f", dados_pwm.indice_modulacao);
    lcd_set_cursor(1, 7);
    lcd_send_string(string_indice_mod);
}
else
{
    dados_pwm.indice_modulacao = 0.9;
    lcd_set_cursor(1, 0);
    lcd_send_string("tensao:   ");
    sprintf(string_indice_mod, "%.2f", dados_pwm.indice_modulacao);
    lcd_set_cursor(1, 7);
    lcd_send_string(string_indice_mod);
}

sprintf(string_freq, "%.2f", freq_inicial);
lcd_set_cursor(0, 0);
lcd_send_string("Freq:     ");
lcd_set_cursor(0, 6);

```

```

    lcd_send_string(string_freq);

    freq_inicial += 1.0;

    if(freq_inicial >= freq_ref){
        mystatus.status_contrl_partd_suave = false;
    }

    vTaskDelay(pdMS_TO_TICKS(100));
}
vTaskDelay(pdMS_TO_TICKS(600));
}
}

void menu(void *pvParameter){
    // static bool status = true;
    static bool stts_bot_esq, stts_bot_drt, stts_bot_cm, stts_bot_bx;
    static bool status_menu_aux1 = true;
    static bool status_menu_aux2;

    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_send_string("menu ok");
    vTaskDelay(1000 / portTICK_PERIOD_MS);

    while (1)
    {
        if (status_menu_aux1)
        {
            lcd_clear();
            lcd_set_cursor(0, 0);
            lcd_send_string("modulacao:  ");
            lcd_set_cursor(1, 0);
            lcd_send_string("SPWM<    SVM");

```

```

stts_bot_esq = true;
while (status_menu_aux1)
{
    if (botao_lido == botao_direita)
    {
        botao_lido = 0;
        stts_bot_drt = true;
        stts_bot_esq = false;

        lcd_set_cursor(1, 4);
        lcd_send_string(" ");
        lcd_set_cursor(1, 12);
        lcd_send_string(">");
    }
    else if (botao_lido == botao_esquerda) // botao esquerdo pressionado
    {
        botao_lido = 0;
        stts_bot_esq = true;
        stts_bot_drt = false; // desativando status do botao direito

        lcd_set_cursor(1, 4);
        lcd_send_string("<");
        lcd_set_cursor(1, 12);
        lcd_send_string(" ");
    }

    // testando quais botoões foram apertados para tomar a decisão do modulador do
PWM
    if (botao_lido == botao_confirma && stts_bot_drt == true)
    {
        // SVM foi selecionado como modulador do pwm

        botao_lido = 0;

```

```

    mystatus.status_modulacao = true; // indica que será usada SVM
    status_menu_aux1 = false;
    status_menu_aux2 = true;

    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_send_string("SVM selecionado");
    vTaskDelay(1000 / portTICK_PERIOD_MS);
}
else if (botao_lido == botao_confirma && stts_bot_esq == true)
{
    // SPWM foi selecionado como modulador do pwm

    botao_lido = 0;
    mystatus.status_modulacao = false; // indica que será usado SPWM
    status_menu_aux1 = false;
    status_menu_aux2 = true;

    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_send_string("SPWM selecionado");
    vTaskDelay(1000 / portTICK_PERIOD_MS);
}

vTaskDelay(400 / portTICK_PERIOD_MS);
} // end while(status_menu_aux1)
} // end if(status_menu_aux1) -- parte 1 do menu finalizado

if (status_menu_aux2)
{
    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_send_string(">analogico  ");
    lcd_set_cursor(1, 0);

```

```

lcd_send_string(" discreto ");

botao_lido = 0;
stts_bot_cm = true;
while (status_menu_aux2)
{
    vTaskDelay(400 / portTICK_PERIOD_MS);

    if (botao_lido == botao_descer)
    {
        botao_lido = 0;
        stts_bot_cm = false;
        stts_bot_bx = true;

        lcd_set_cursor(0, 0);
        lcd_send_string(" ");
        lcd_set_cursor(1, 0);
        lcd_send_string(">");
    }
    else if (botao_lido == botao_subir)
    {
        botao_lido = 0;
        stts_bot_cm = true;
        stts_bot_bx = false;

        lcd_set_cursor(0, 0);
        lcd_send_string(">");
        lcd_set_cursor(1, 0);
        lcd_send_string(" ");
    } // end else if

    if (botao_lido == botao_confirma && stts_bot_cm == true)
    {

```

```

    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_send_string("iniciando ");
    lcd_set_cursor(0, 0);
    lcd_send_string("partida suave");

    // controle de frequência pelo potenciômetro ativado
    status_menu_aux2 = false;
    botao_lido = 0;
    vTaskDelay(500 / portTICK_PERIOD_MS);

    mystatus.status_timer_mcpwm = true;
    mystatus.status_contrl_partd_suave = true;
    mystatus.status_contrl_discreto = false;
    mystatus.status_contrl_pot = true;
}
else if (botao_lido == botao_confirma && stts_bot_bx == true)
{
    // partida suave ativado
    status_menu_aux2 = false;
    botao_lido = 0;
    vTaskDelay(500 / portTICK_PERIOD_MS);

    mystatus.status_timer_mcpwm = true;
    mystatus.status_contrl_partd_suave = true;
    mystatus.status_contrl_discreto = true;
    mystatus.status_contrl_pot = false;
} // end else if

vTaskDelay(400 / portTICK_PERIOD_MS);
} // end while(status_menu_aux_2)
} // end if(status_menu_aux2)

vTaskDelay(400 / portTICK_PERIOD_MS);

```

```

    } // end while(1)
}

void ler_botoes(void *pvParameter)
{
    // inicializando os botões como entrada
    for (uint8_t i = 0; i < 8; i++)
    {
        esp_rom_gpio_pad_select_gpio(botoes[i]);
        gpio_set_direction(botoes[i], GPIO_MODE_INPUT);
        gpio_pulldown_dis(botoes[i]);
        gpio_pullup_dis(botoes[i]);
    }

    while (1)
    {
        for (uint8_t i = 0; i < 8; i++)
        {
            if (gpio_get_level(botoes[i]) == 1)
            {
                botao_lido = botoes[i];
            }
        }
    }

    vTaskDelay(pdMS_TO_TICKS(50)); // delay de 50ms para debouncing dos botões
}

void task1(void *pvParameter)
{
    static bool status_led = 0;
    while (1)
    {
        status_led = !status_led;
    }
}

```



```

    gpio_set_level(GPIO_NUM_9, status_led);
    vTaskDelay(pdMS_TO_TICKS(1000));
}
}

/*
=====
                        CONFIGURAÇÃO DO MCPWM
=====
*/

void inicia_mcpwm()
{
    ESP_LOGI(TAG, "criando MCPWM timer");
    mcpwm_timer_config_t timer_config = {
        .group_id = 0,
        .clk_src = MCPWM_TIMER_CLK_SRC_DEFAULT,
        .resolution_hz = freq_resolution_mcpwm,
        .count_mode = MCPWM_TIMER_COUNT_MODE_UP_DOWN,
        .period_ticks = dados_pwm.amplitude_senoide,
    };
    ESP_ERROR_CHECK(mcpwm_new_timer(&timer_config, &timer));

    ESP_LOGI(TAG, "criando MCPWM operator");
    mcpwm_oper_handle_t operators[3];
    mcpwm_operator_config_t operator_config = {
        .group_id = 0,
    };
    for (int i = 0; i < 3; i++)
    {
        ESP_ERROR_CHECK(mcpwm_new_operator(&operator_config, &operators[i]));
    }

    ESP_LOGI(TAG, "conectando operadores ao mesmo timer");

```

```

for (int i = 0; i < 3; i++)
{
    ESP_ERROR_CHECK(mcpwm_operator_connect_timer(operators[i], timer));
}

ESP_LOGI(TAG, "criando comparadores");
mcpwm_cmpr_handle_t comparators[3];
mcpwm_comparator_config_t compare_config = {
    .flags.update_cmp_on_tez = true,
    .flags.update_cmp_on_tep = true,
};
for (int i = 0; i < 3; i++)
{
    ESP_ERROR_CHECK(mcpwm_new_comparator(operators[i], &compare_config,
&comparators[i]));

    ESP_ERROR_CHECK(mcpwm_comparator_set_compare_value(comparators[i], 0)); //
dados_pwm.amplitude_senoide / 2
}

ESP_LOGI(TAG, "criando MCPWM generators");
mcpwm_gen_handle_t generators[3][2] = {};
mcpwm_generator_config_t gen_config = {};
const int gen_gpios[3][2] = {
    {fase_A_H, fase_A_L},
    {fase_B_H, fase_B_L},
    {fase_C_H, fase_C_L}};
for (int i = 0; i < 3; i++)
{
    for (int j = 0; j < 2; j++)
    {
        gen_config.gen_gpio_num = gen_gpios[i][j];
        ESP_ERROR_CHECK(mcpwm_new_generator(operators[i], &gen_config,
&generators[i][j]));
    }
}

```

```

    }
}

ESP_LOGI(TAG, "setando acoes dos geradores");
for (int i = 0; i < 3; i++)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_actions_on_compare_event(generators[i]
[0],
    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_
UP, comparators[i], MCPWM_GEN_ACTION_HIGH),
    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION
_DOWN, comparators[i], MCPWM_GEN_ACTION_LOW),
    MCPWM_GEN_COMPARE_EVENT_ACTION_END()));
}

ESP_LOGI(TAG, "configurando dead time");
mcpwm_dead_time_config_t config_tempo_morto = {
    .posedge_delay_ticks = 80,
    .negedge_delay_ticks = 0,
    .flags.invert_output = true
};
for (int i = 0; i < 3; i++)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(generators[i][0],
generators[i][0], &config_tempo_morto));
}

// saídas do PWM A e B serão complementares entre si
config_tempo_morto = (mcpwm_dead_time_config_t){
    .posedge_delay_ticks = 0,
    .negedge_delay_ticks = 80,
    .flags.invert_output = false,
};

```

```

for (int i = 0; i < 3; i++)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(generators[i][0],
generators[i][1], &config_tempo_morto));
}

ESP_LOGI(TAG, "habilitando interrupção por timer para ajustar o duty cycle");
esp_timer_handle_t periodic_timer = NULL;
const esp_timer_create_args_t args_periodic_timers = {
    .callback = mcpwm_isr_timer,
    .arg = comparators,
};
ESP_ERROR_CHECK(esp_timer_create(&args_periodic_timers, &periodic_timer));
ESP_ERROR_CHECK(esp_timer_start_periodic(periodic_timer, 50));

ESP_LOGI(TAG, "iniciando o MCPWM_TIMER");
ESP_ERROR_CHECK(mcpwm_timer_enable(timer));
    ESP_ERROR_CHECK(mcpwm_timer_start_stop(timer,
MCPWM_TIMER_START_NO_STOP));

printf("MCPWM configurado\n");
}

// TEMPO MORTO ATIVO BAIXO COMPLEMENTAR

/*
ESP_LOGI(TAG, "configurando dead time");
mcpwm_dead_time_config_t config_tempo_morto = {
    .posedge_delay_ticks = 80,
    .negedge_delay_ticks = 0,
    .flags.invert_output = true
};
for (int i = 0; i < 3; i++)
{

```

```

        ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(generators[i][0],
generators[i][0], &config_tempo_morto));
    }

    // saídas do PWM A e B serão complementares entre si
    config_tempo_morto = (mcpwm_dead_time_config_t){
        .posedge_delay_ticks = 0,
        .negedge_delay_ticks = 80,
        .flags.invert_output = false,
    };
    for (int i = 0; i < 3; i++)
    {
        ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(generators[i][0],
generators[i][1], &config_tempo_morto));
    }
    */

    // TEMPO MORTO ATIVO ALTO COMPLEMENTAR

    /*
    ESP_LOGI(TAG, "configurando dead time");
    mcpwm_dead_time_config_t config_tempo_morto = {
        .posedge_delay_ticks = 80,
        .negedge_delay_ticks = 0,
    };
    for (int i = 0; i < 3; i++)
    {
        ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(generators[i][0],
generators[i][0], &config_tempo_morto));
    }

    // saídas do PWM A e B serão complementares entre si
    config_tempo_morto = (mcpwm_dead_time_config_t){
        .posedge_delay_ticks = 0,

```

```
.negedge_delay_ticks = 80,  
.flags.invert_output = true,  
};  
for (int i = 0; i < 3; i++)  
{  
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(generators[i][0],  
generators[i][1], &config_tempo_morto));  
}  
*/
```

ANEXO B – Parte 1 da biblioteca criada para o código do algoritmo do SVM e SPWM

Biblioteca “gerador_senoide.h”:

```
#include "esp_err.h"
#include "esp_log.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "rom/ets_sys.h"
#include "math.h"

typedef struct {
    float Ualpha;    // Input: reference alpha-axis phase voltage
    float Ubeta;     // Input: reference beta-axis phase voltage
    float Ta;        // Output: reference phase-a switching function
    float Tb;        // Output: reference phase-b switching function
    float Tc;        // Output: reference phase-c switching function
    float tmp1;      // Variable: temp variable
    float tmp2;      // Variable: temp variable
    float tmp3;      // Variable: temp variable
    unsigned int VecSector; // Space vector sector
} svm_t;

// ----- //

typedef struct{
    int  amplitude_senoide;    //valor de estouro do timer
    int  num_amostras;
    int  frequencia_modulacao;
    float periodo_modulacao;
    float freq_senoide;
    float freq_inicial;
    float indice_modulacao;
    float ohmega;              //2 . PI . f . periodo_modulacao
    float angulo_fase;
} dados_modulacao_t;
```

```
// ----- //
```

```
typedef struct{  
    float seno;  
    float cosseno;  
    //float seno_3h;  
    int valor_fase_A;  
    int valor_fase_B;  
    int valor_fase_C;  
} pwm_senoidal_t;
```

```
// ----- //
```



```
void svm_run(svm_t *v);  
void spwm_gen(dados_modulacao_t *dados, pwm_senoidal_t *pwm, int contador);
```


ANEXO C - Parte 2 da biblioteca criada para o código do algoritmo do SVM e SPWM

Funções da biblioteca “gerador_senoide.h”:

```
#include <stdio.h>
#include "gerador_senoide.h"

// ----- //

void svm_run(svm_t *v){
    v->tmp1= v->Ubeta;
    v->tmp2= (0.5 * v->Ubeta) + (0.866 * v->Ualpha);
    v->tmp3= v->tmp2 - v->tmp1;

    v->VecSector=3;
    v->VecSector=(v->tmp2> 0)?(v->VecSector-1):v->VecSector;
    v->VecSector=(v->tmp3> 0)?(v->VecSector-1):v->VecSector;
    v->VecSector=(v->tmp1< 0)?(7-v->VecSector):v->VecSector;

    if(v->VecSector==1 || v->VecSector==4) {
        v->Ta = v->tmp2;
        v->Tb= v->tmp1 - v->tmp3;
        v->Tc=-v->tmp2;
    }

    else if(v->VecSector==2 || v->VecSector==5) {
        v->Ta= v->tmp3+v->tmp2;
        v->Tb= v->tmp1;
        v->Tc=-v->tmp1;
    }

    else {
        v->Ta= v->tmp3;
        v->Tb=-v->tmp3;
        v->Tc=-(v->tmp1+v->tmp2);
    }
}
```

```
}
```

```
#define raiz_de_3 1.73205080757
```

```
void spwm_gen(dados_modulacao_t *dados, pwm_senoidal_t *pwm, int contador){
```

```
    dados->angulo_fase = dados->ohmega * contador;
```

```
    pwm->seno = sin(dados->angulo_fase);
```

```
    pwm->cosseno = cos(dados->angulo_fase);
```

```
    //cosseno = sqrt(1 - (seno) * (seno));
```

```
    //senoide terceira harmônica:
```

```
    //seno = sin(3 * dados->ohmega * contador);
```

```
    // ondas senoidais defasadas 120° entre si
```

```
    // calcular seno = sen(theta)
```

```
    // cosseno = cos(theta) ou cosseno = sqrt(1 - (seno)**2)
```

```
    // sen(theta - 2pi/3) = -sen(theta)/2 - sqrt(3)*cos(theta)/2
```

```
    // sen(theta - 2pi/3) = -sen(theta)/2 + sqrt(3)*cos(theta)/2
```

```
    //por algum motivo contador timer é metade do que realmente deveria ser, por tanto divisor  
    por 4
```

```
    pwm->valor_fase_A = dados->amplitude_senoide * 0.25 * (1 + dados->indice_modulacao *  
pwm->seno);
```

```
    pwm->valor_fase_B = dados->amplitude_senoide * 0.25 * (1 + dados->indice_modulacao *  
(-pwm->seno * 0.5 - raiz_de_3 * pwm->cosseno * 0.5));
```

```
    pwm->valor_fase_C = dados->amplitude_senoide * 0.25 * (1 + dados->indice_modulacao *  
(-pwm->seno * 0.5 + raiz_de_3 * pwm->cosseno * 0.5));
```

```
}
```

```
// ----- //
```